

Using AI To Classify Radio Frequency Emanations

UT Dallas Team 2

Adithya Chintala – Student Researcher – Adithya.Chintala@UTDallas.edu

Brigham Thornock – Student Researcher – Brigham.Thornock@UTDallas.edu

Ian Zhang – Student Researcher – ian.zhang@utdallas.edu

Ryan Nelson – Student Researcher – Ryan.Nelson@UTDallas.edu

Samuel Ofiaza – Student Researcher – Samuel.Ofiaza@UTDallas.edu

Brian Ricks – Faculty Support – absolutefunk@utdallas.edu

Problem Mentors:

Brad Kline - National Security Agency - bjkline@uwe.nsa.gov

Drew Wilson - National Security Agency - aewilso@uwe.nsa.gov

Sam Osofsky - National Security Agency - stosofs@radium.ncsc.mil

Investigator Biographies

Adithya Chintala:

MS in CS (Cyber). SFS. INL Intern.

Focus: Anomaly Detection, Adversarial AI.

Ian Zhang:

MS in CS (AI/ML). Toyota & F5 Intern.

Focus: Crypto, AI/ML Applications.

Brigham Thornock:

MS in CS (Cyber). SFS, IDA Intern.

Focus: ML, Offensive Security.

Samuel Ofiaza:

MS in CS (Cyber). SFS, INL Intern.

Focus: Adversarial ML.

Ryan Nelson:

MS in CS (Cyber). SFS, LANL Intern.

Focus: Hardware security, RF, and ML.

Agenda

- Investigator Biographies
- Project Introduction
- Project Objectives
- Research Plan and Data Overview
- Model Approaches and Key Findings
- Data Issues and Project Challenges
- Conclusions and Future Work

Project Introduction

The Problem:

- Electronic systems emit unintended Radio Frequency (RF) signals.
- Adversaries can intercept these signals to reconstruct sensitive data.
- This attack bypasses traditional encryption and software defenses by targeting the physical layer.

Our Research Question:

Can AI/ML models reliably identify and classify data patterns (e.g. characters) from these RF emanations?

We aim to build an AI pipeline to demonstrate this vulnerability and explore its limits.

Project Objectives

Primary Goal:

Develop a reliable AI pipeline for character classification from RF signals.

Specific Objectives:

- Build a robust data processing pipeline for I/Q samples.
- Develop and optimize multiple ML architectures (CNN, RNN, etc.)
- Test models on progressively difficult datasets.
- Create a final, high-performance ensemble model.

Research Plan and Progress

- **Data:**

- Template pickle files: contains a “snapshot” of the signal of specific characters/bytes (Ex. signal for the character ‘A’)
 - Unpickled and converted to dataframes (Pandas/Polars) for easier workability
- Sequence pickle files: contains a signal of a series of characters/block of text

- **Models Considered:**

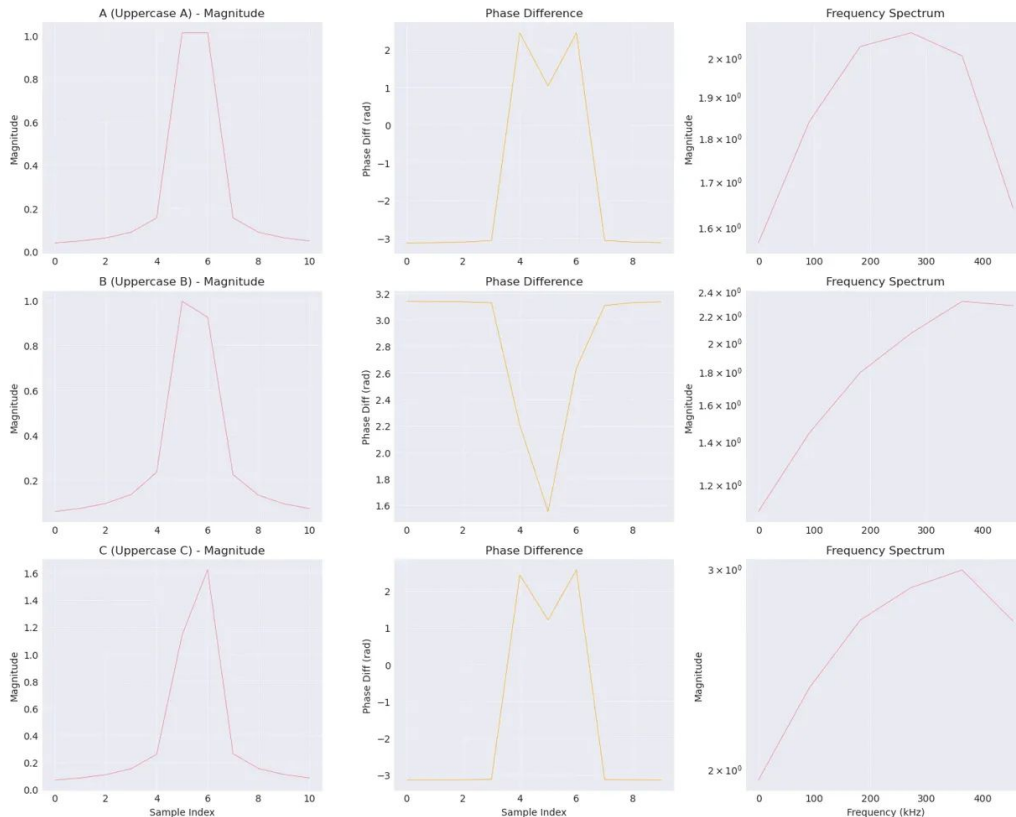
- Deep Learning Models: CNNs, LSTMs, MLPs
- Traditional Supervised Models: SVMs (Support Vector Machines), Random Forest, Naive Bayes, Decision Trees, KNN (K-Nearest Neighbors), GBM (Gradient Boosting Machines)

- **Progress:** Preprocessed the data and tested these models on the template and sequence files

Part 0: Data Visualization

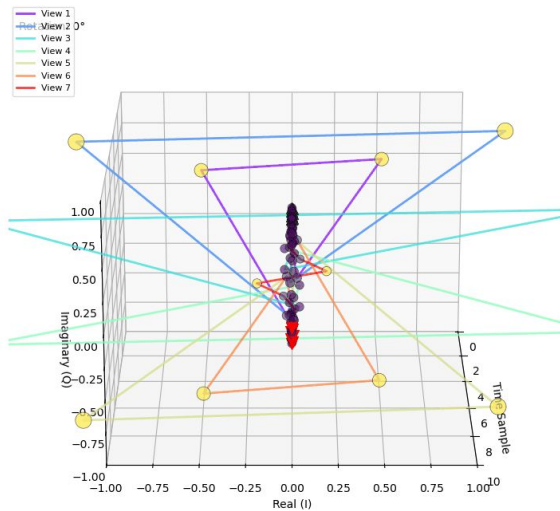
Dataset Overview

- **Data Type:** Complex-valued RF emanations from computers processing text (unintended electromagnetic leakage)
- **Signal Structure:** 11-sample complex signals per character, sampled at 1 MHz
- **Multi-view Recording:** 7 simultaneous RF receivers at different spatial positions capturing the same emanation
- **Dataset Scale:** 128 ASCII characters $\times$ 7 views $\times$ 4 recordings = 3,584 samples per file
- **Noise Conditions:** 5 levels (NL00-NL04) ranging from clean to 30% bit error rate
- **Character Density:** 3 padding levels (CD00-CD02) with 0, 4, or 10 NULL symbols between characters



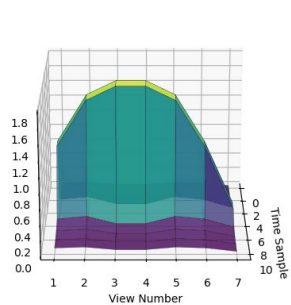
Part 0: Data Visualization

Letter 'A' - 3D RF Emission Signature
7 Receiver Views

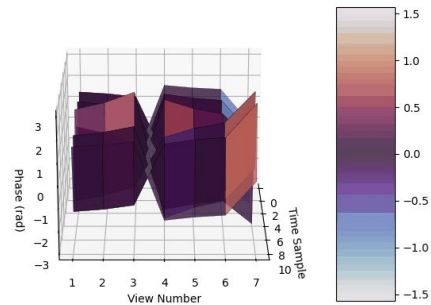


Letter 'A' - Multi-View RF Emission Analysis
Rotation: 0°

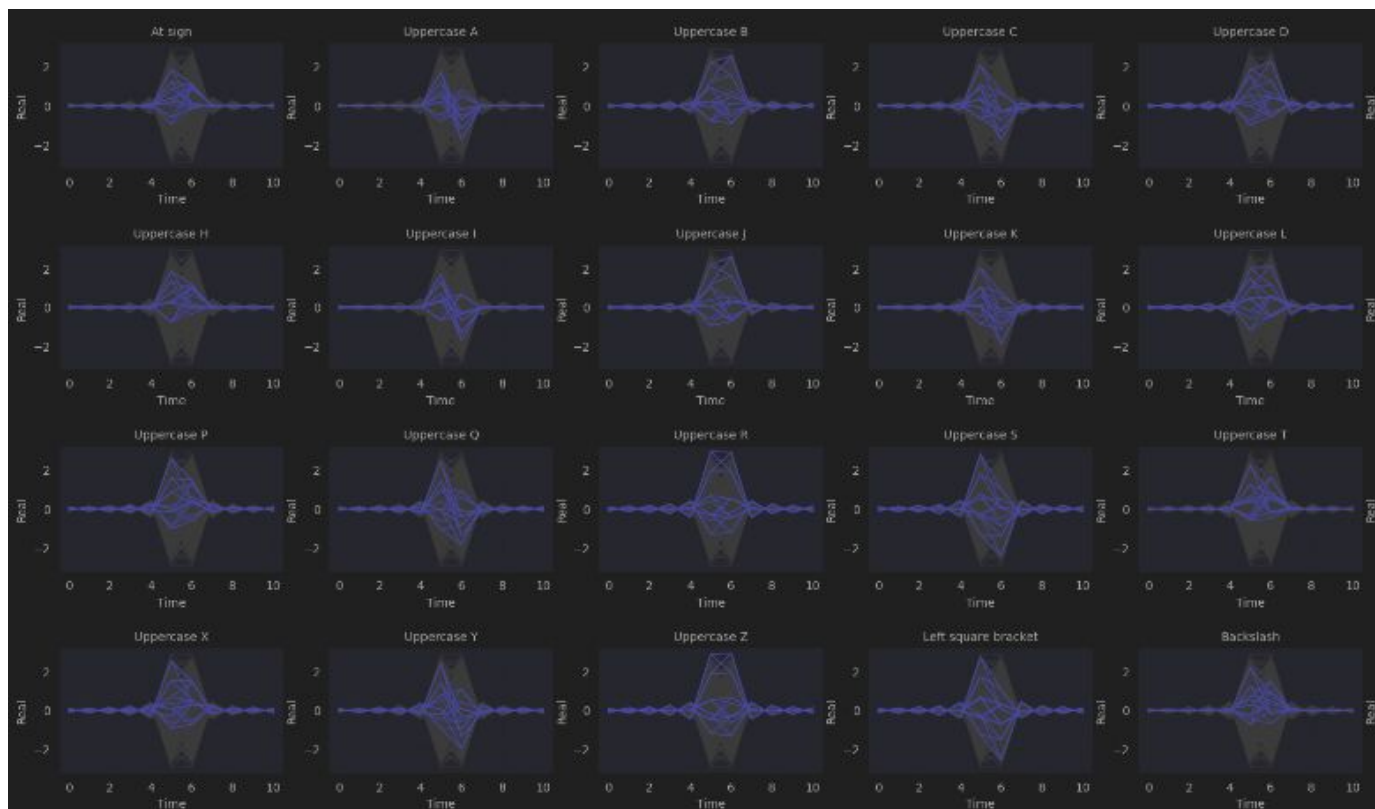
Magnitude Surface Across Views



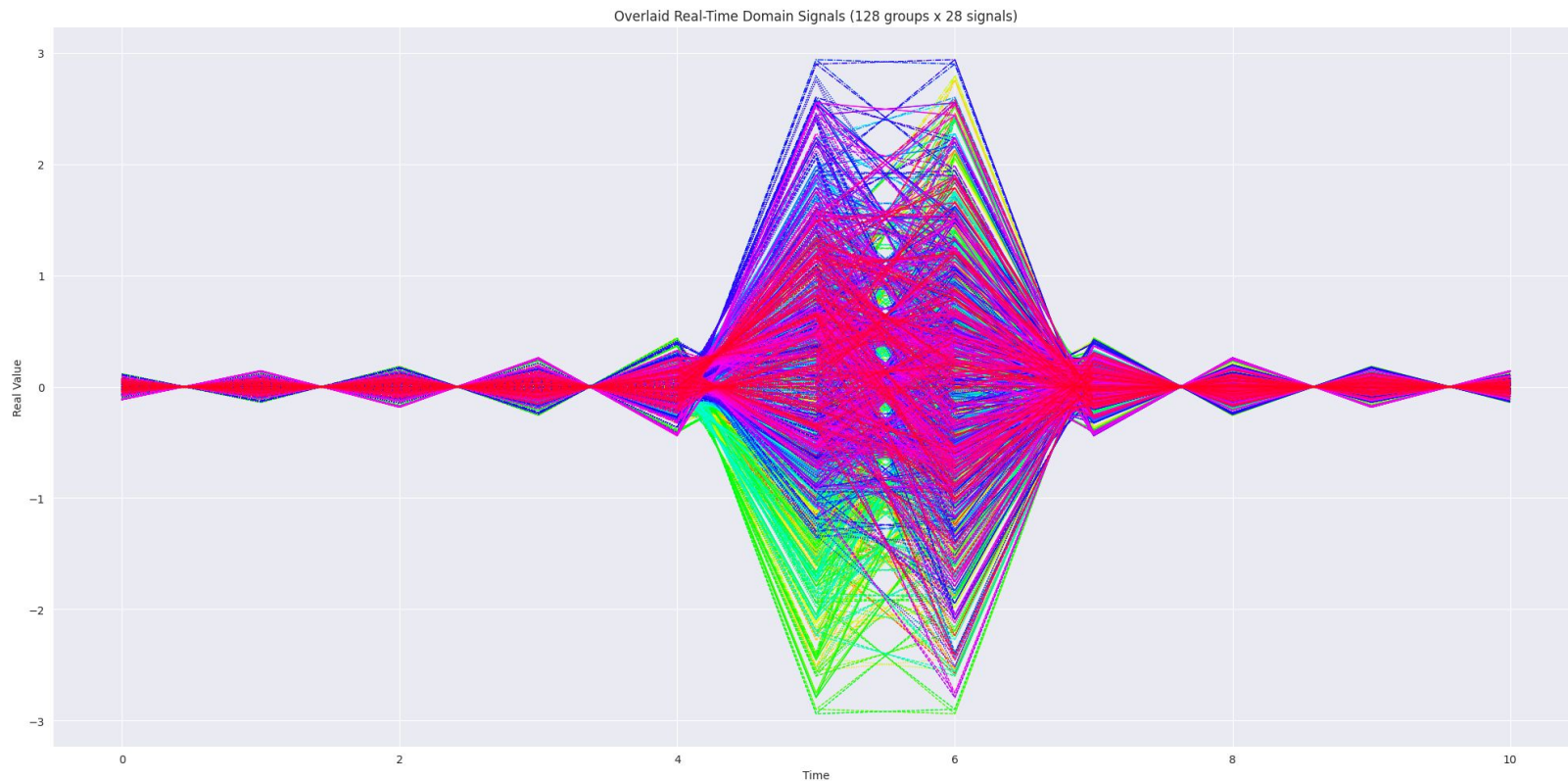
Phase Surface Across Views



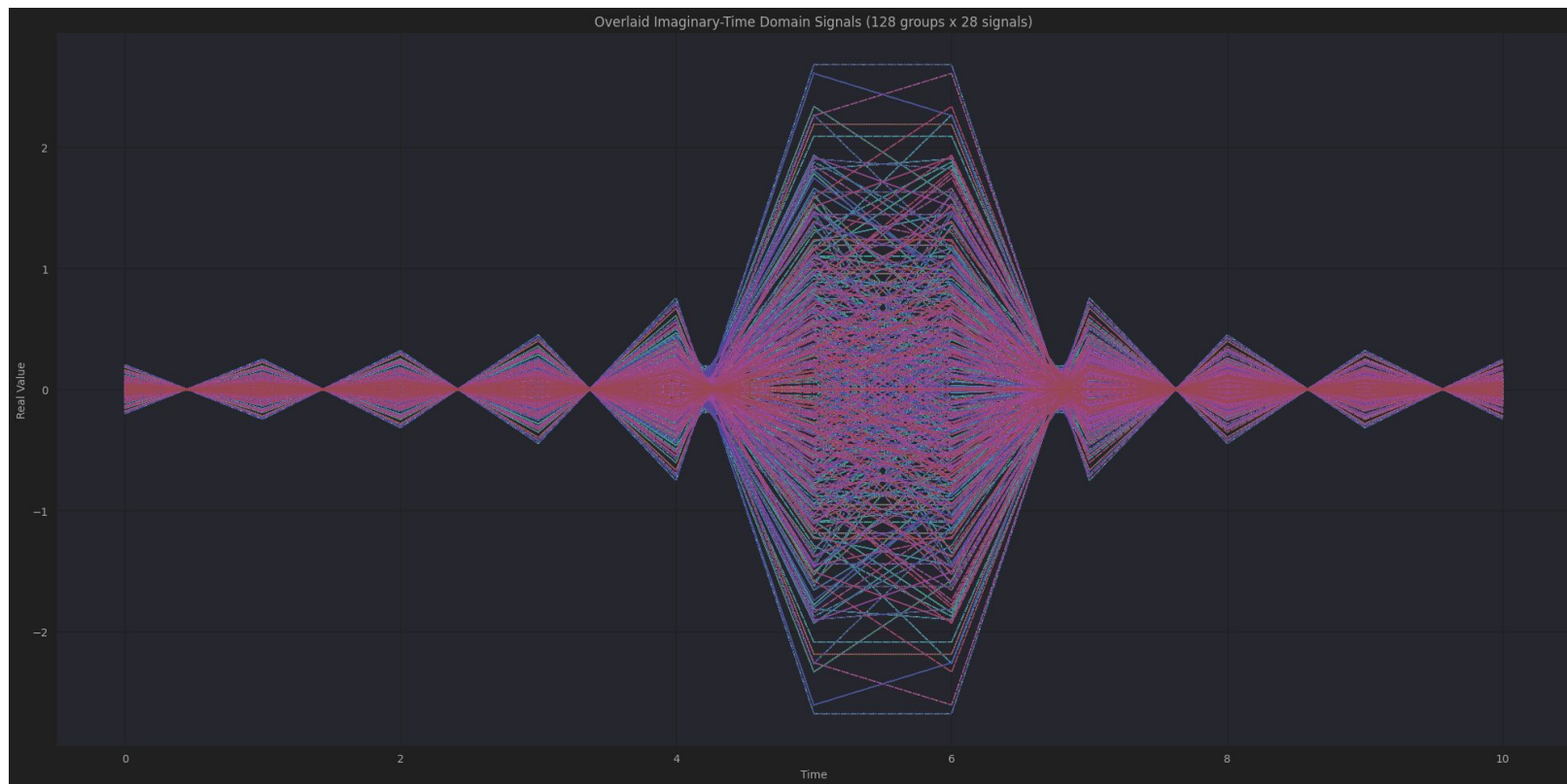
Template Visualization: Character vs All



Template Visualization: All Overlaid (Real)

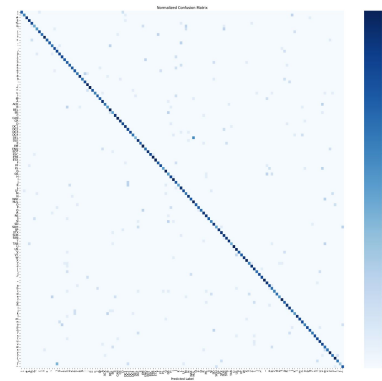
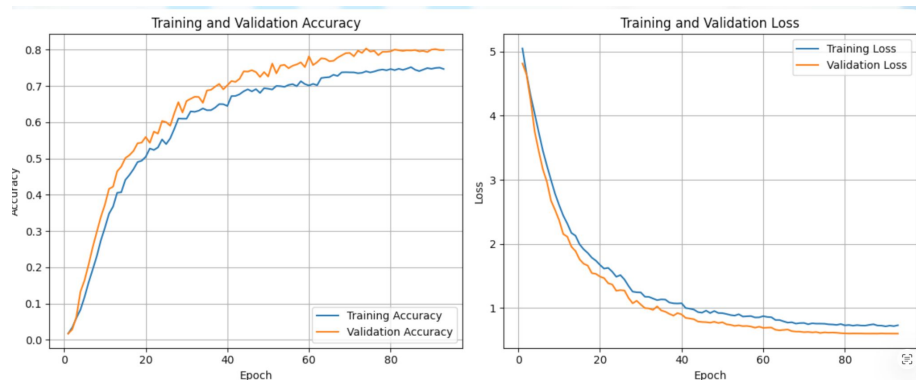


Template Visualization: All Overlaid (Imaginary)



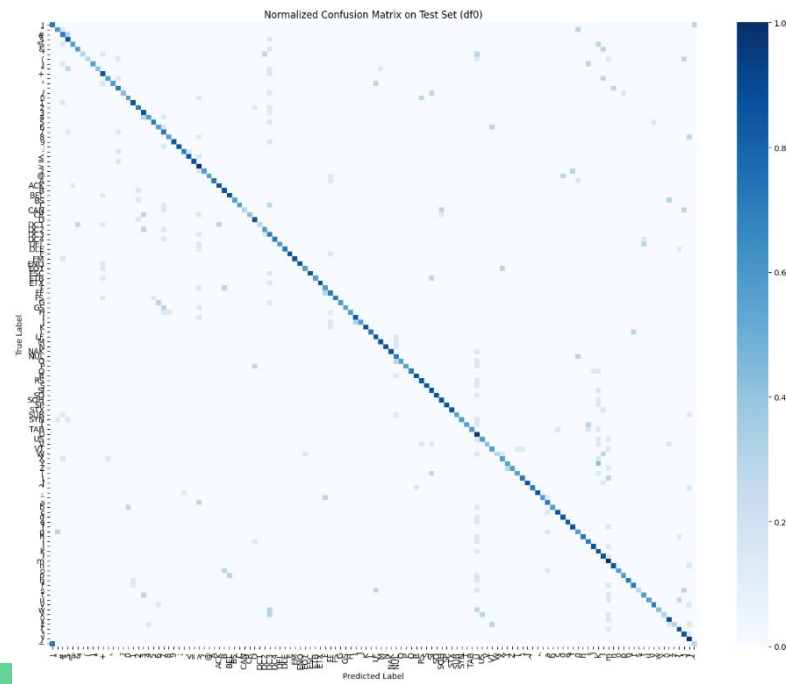
Part 1: CNN Approach

- Ideal candidate for detecting patterns in the context
- Input: 11 (padding + character) * 2 (in-phase and quadrature components)
- Output: Multi-class classifier
- Result: 0.80 accuracy, 0.82 precision, 0.80 recall, and 0.80 F1 score
- Inconsistent result for individual characters: 0.44 - 1.00 accuracy



Part 2: The LSTM + MLP Hybrid Approach

- LSTM (Long Short-Term Memory): good for sequential data
- MLP (Multi-Layer Perceptron): good for static/categorical data
- Hybrid approach: best of both worlds
- Result: ~68% accuracy
- Takeaways:
 - Better hyperparameters
 - Better training/testing methodology
 - Model wasn't the best fit?



Part 3.1: Multilayer Perceptron (MLP)

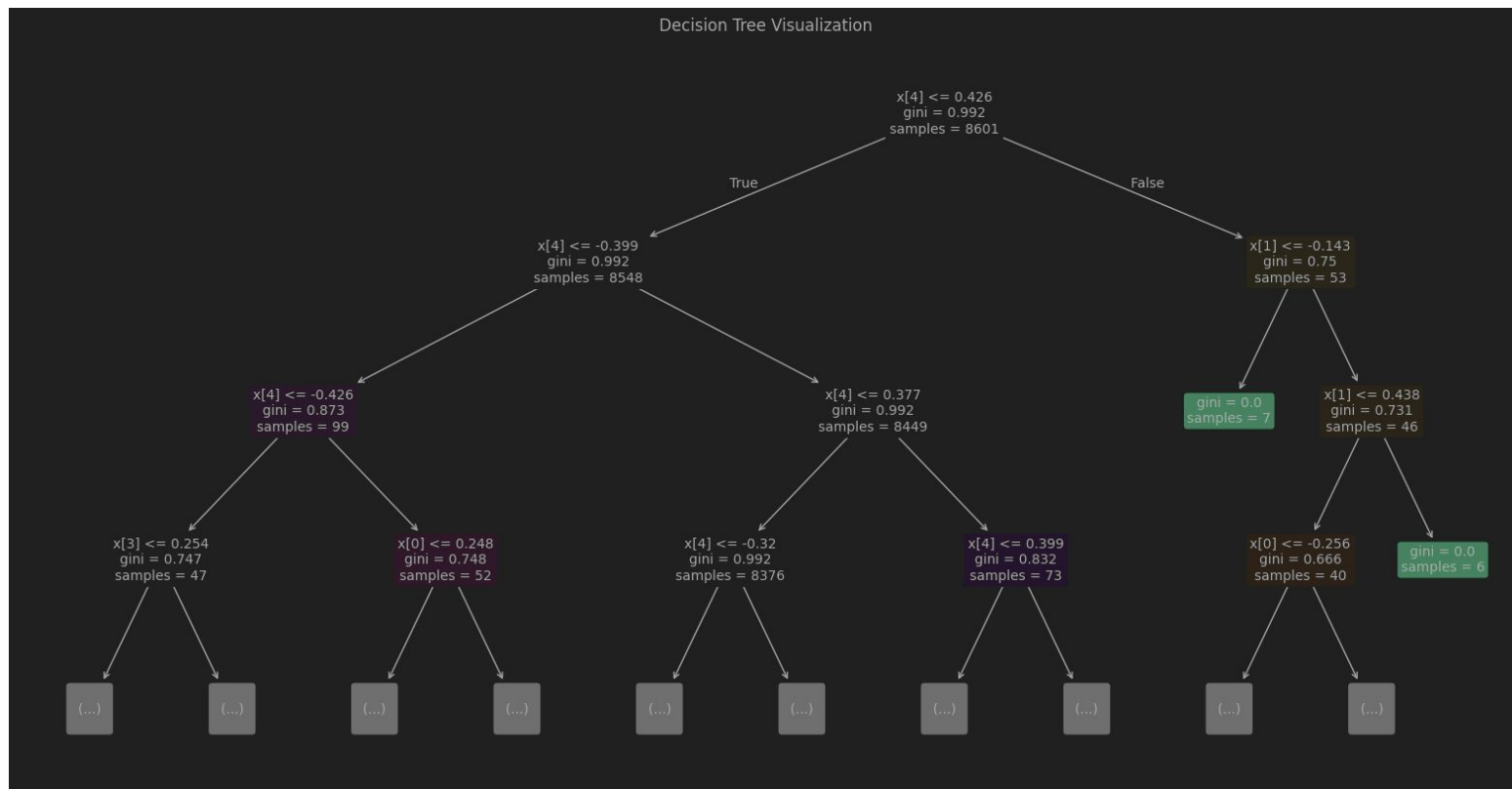
Layer (type:depth-idx)	Input Shape	Output Shape	Param #
Sequential	[1, 22]	[1, 128]	--
└Linear: 1-1	[1, 22]	[1, 512]	11,776
└ReLU: 1-2	[1, 512]	[1, 512]	--
└Dropout: 1-3	[1, 512]	[1, 512]	--
└Linear: 1-4	[1, 512]	[1, 512]	262,656
└ReLU: 1-5	[1, 512]	[1, 512]	--
└Dropout: 1-6	[1, 512]	[1, 512]	--
└Linear: 1-7	[1, 512]	[1, 256]	131,328
└ReLU: 1-8	[1, 256]	[1, 256]	--
└Dropout: 1-9	[1, 256]	[1, 256]	--
└Linear: 1-10	[1, 256]	[1, 256]	65,792
└ReLU: 1-11	[1, 256]	[1, 256]	--
└Dropout: 1-12	[1, 256]	[1, 256]	--
└Linear: 1-13	[1, 256]	[1, 128]	32,896
Total params: 504,448			
Trainable params: 504,448			
Non-trainable params: 0			
Total mult-adds (Units.MEGABYTES): 0.50			
Input size (MB): 0.00			
Forward/backward pass size (MB): 0.01			
Params size (MB): 2.02			
Estimated Total Size (MB): 2.03			

	precision	recall	f1-score
!	0.928571	1.000000	0.962963
"	0.900000	1.000000	0.947368
#	0.689655	0.833333	0.754717
\$	1.000000	0.535714	0.697674
%	0.619048	1.000000	0.764706
...	...	...	...
}	0.923077	0.705882	0.800000
~	0.720000	0.818182	0.765957
accuracy	0.830233	0.830233	0.830233
macro avg	0.849612	0.839277	0.832103
weighted avg	0.855835	0.830233	0.830592

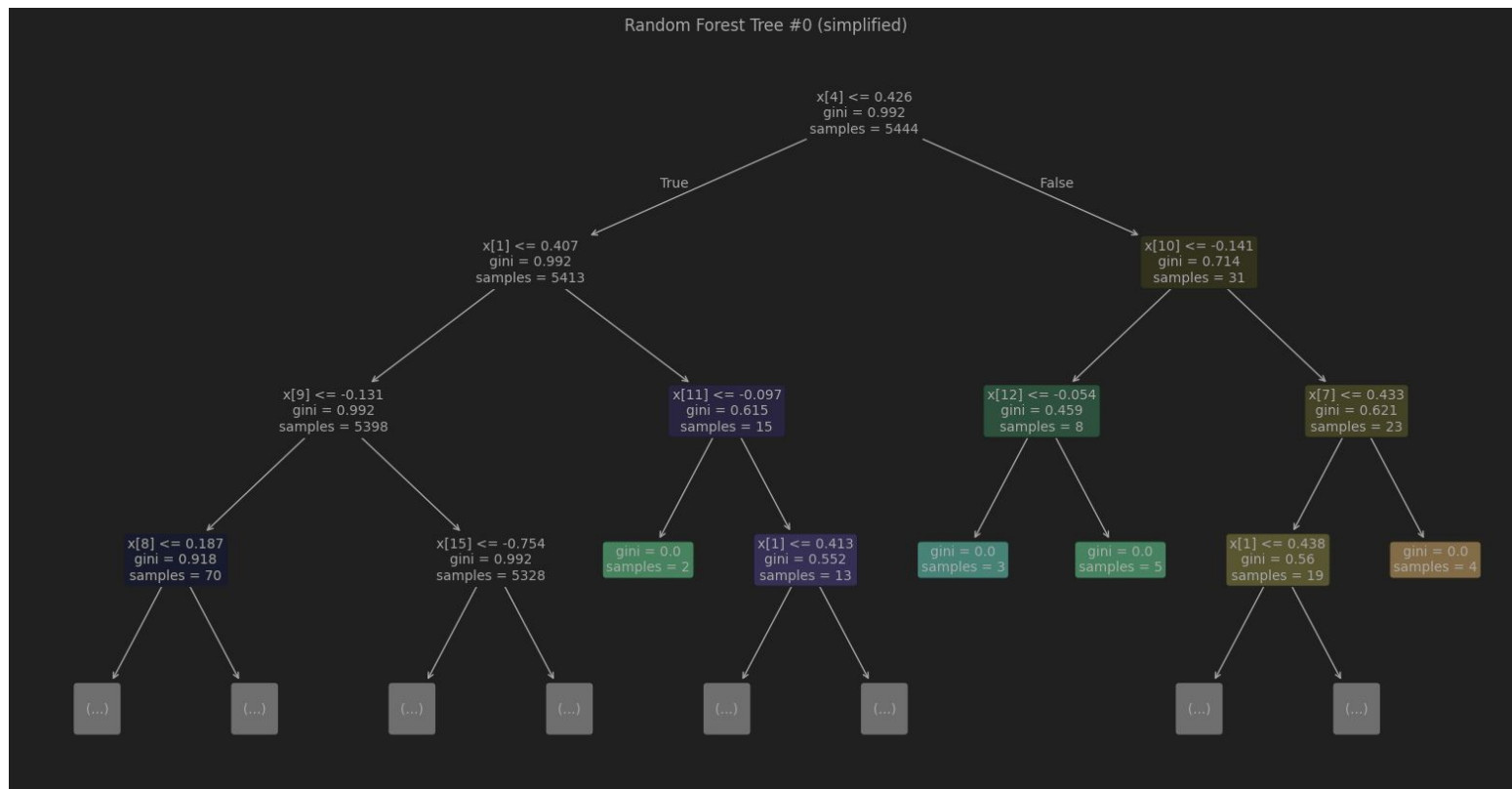
Part 3.2: Traditional Supervised Learning Models

Supervised Classification Model	Real/Imaginary Raw Padded	Magnitude/Phase Raw Padded
Naive Bayes	0.04	0.02
Decision Tree	0.97	0.94
Random Forest	0.99	0.95
SVM	0.25	0.20
KNN	0.20	0.28
XGBoost	0.96	0.95

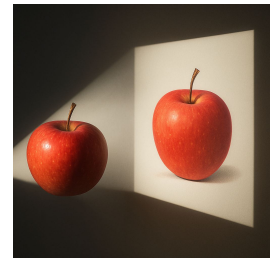
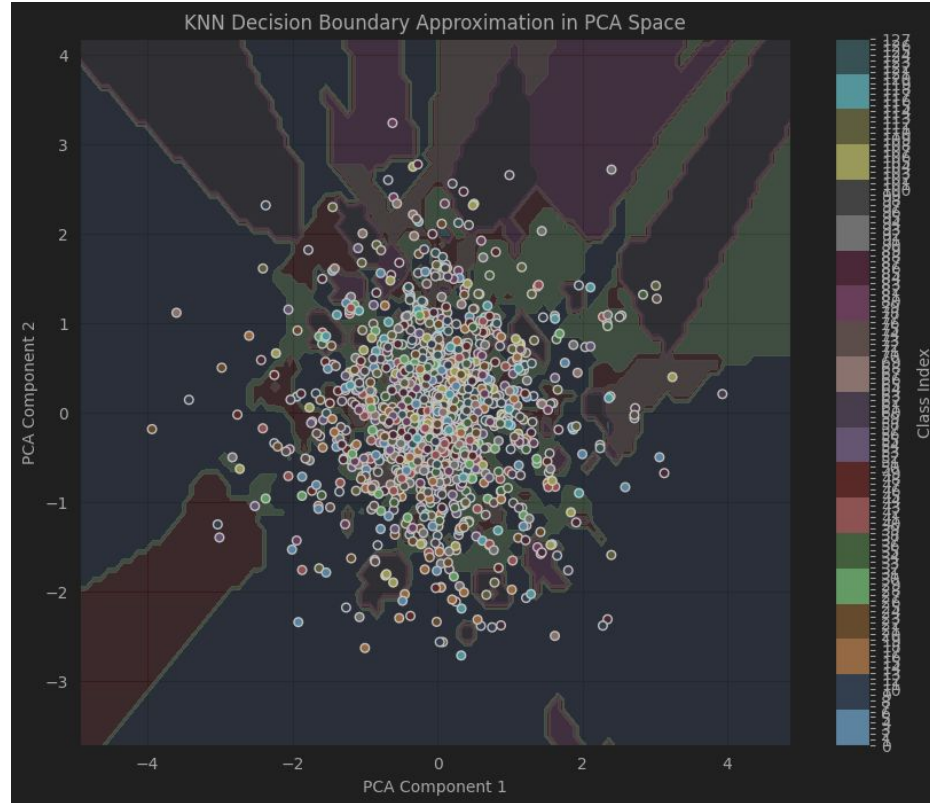
Decision Tree Visualization



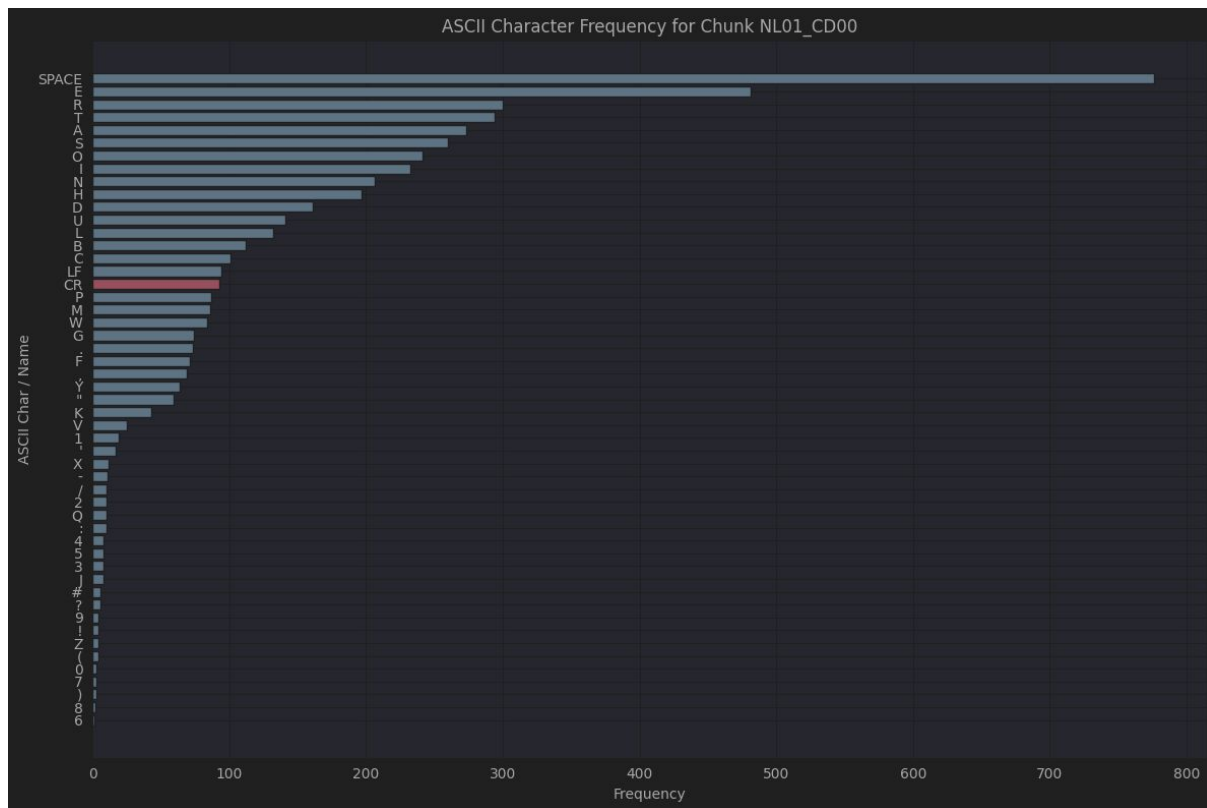
Random Forest Visualization



K-Nearest Neighbors: Decision Boundaries



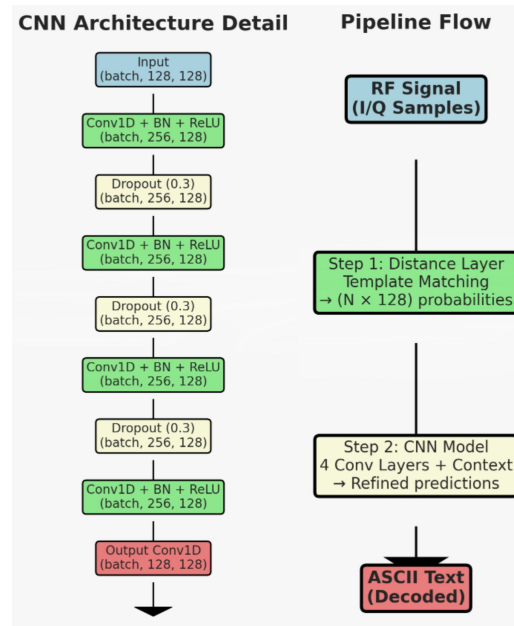
Sequence-Chunk Mismatch: Carriage Returns



Part 4: Classifying Sequences using Distance Match + CNN

Two step approach:

- Step 1: Distance Layer
 - Comparing templates to sequences
- Step 2: 1D-CNN Sequence Model
 - Using comparisons and context to make predictions



Distance Match + CNN Results

Tested on every combination:

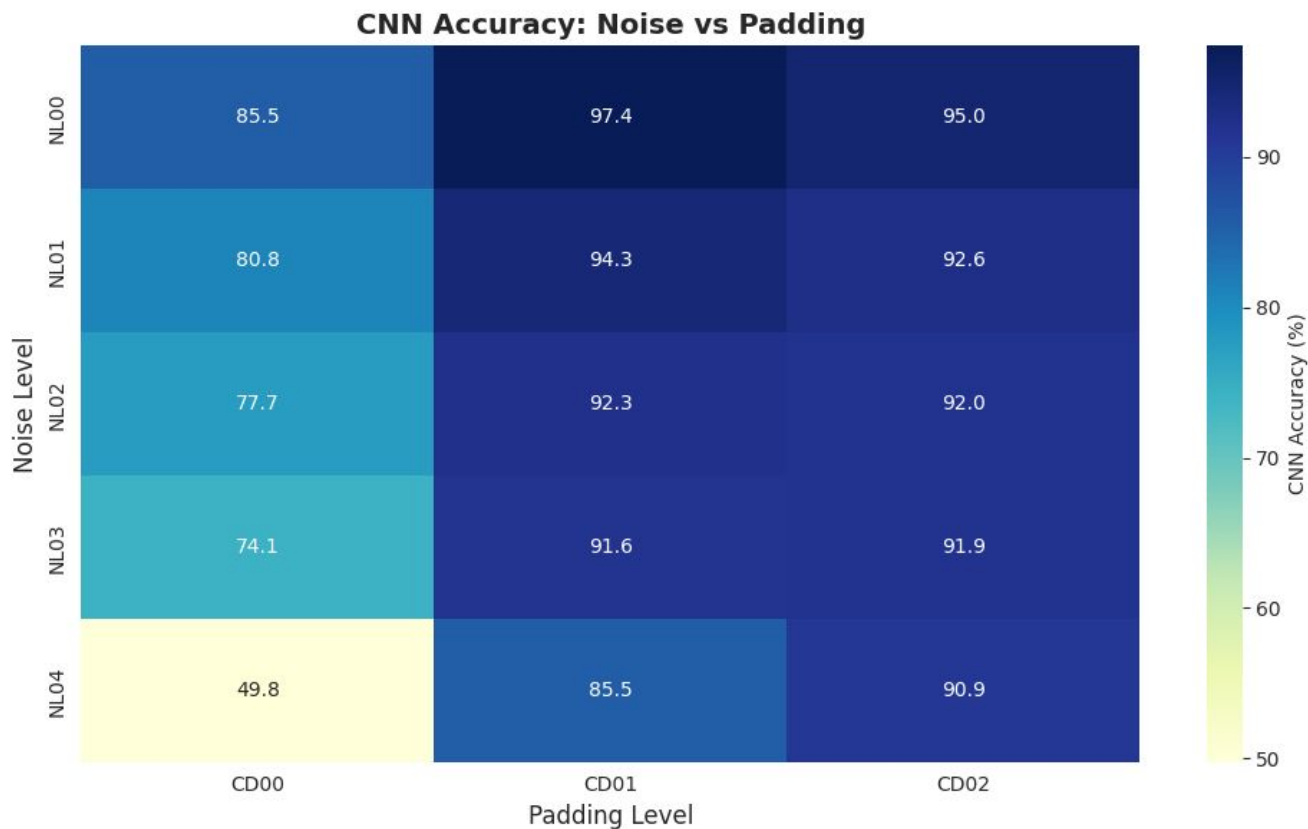
- 5 different noise levels (0%, 1%, 5%, 10%, 30% BER)
- 3 levels of padding (0, 4, 10 null characters)
- 127 view combinations
 - Powerset of 7 views (not including empty set)

```
=====
CNN ACCURACY SUMMARY
=====

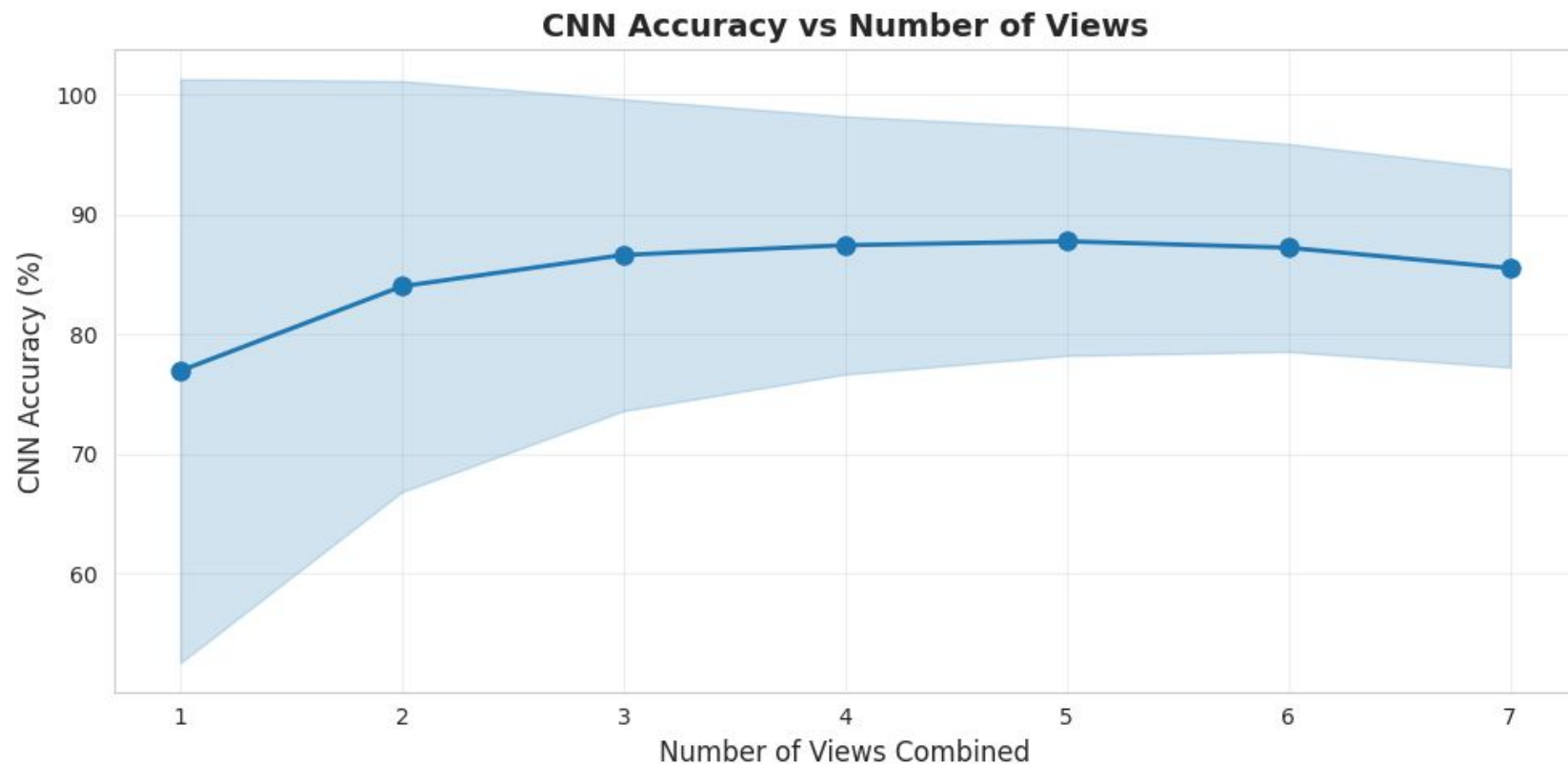
Total experiments: 1905
Successful: 1905 (100.0%)
Failed: 0

Metric                Mean          Std          Min          Max
-----
CNN Accuracy (Test Set) 86.10%       13.61%       8.51%       99.98%
```

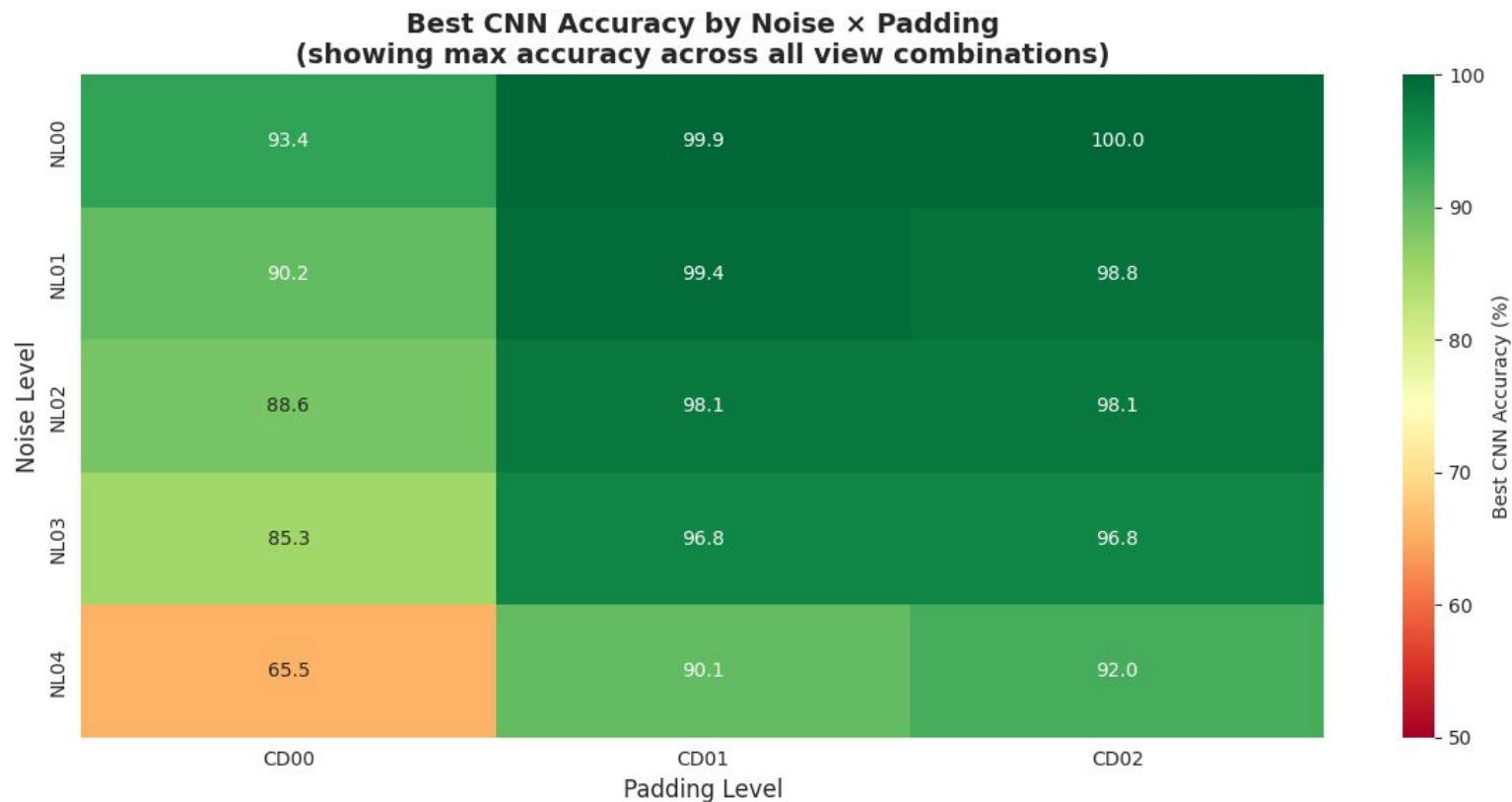
Distance Match + CNN Results cont.



Distance Match + CNN Results cont.



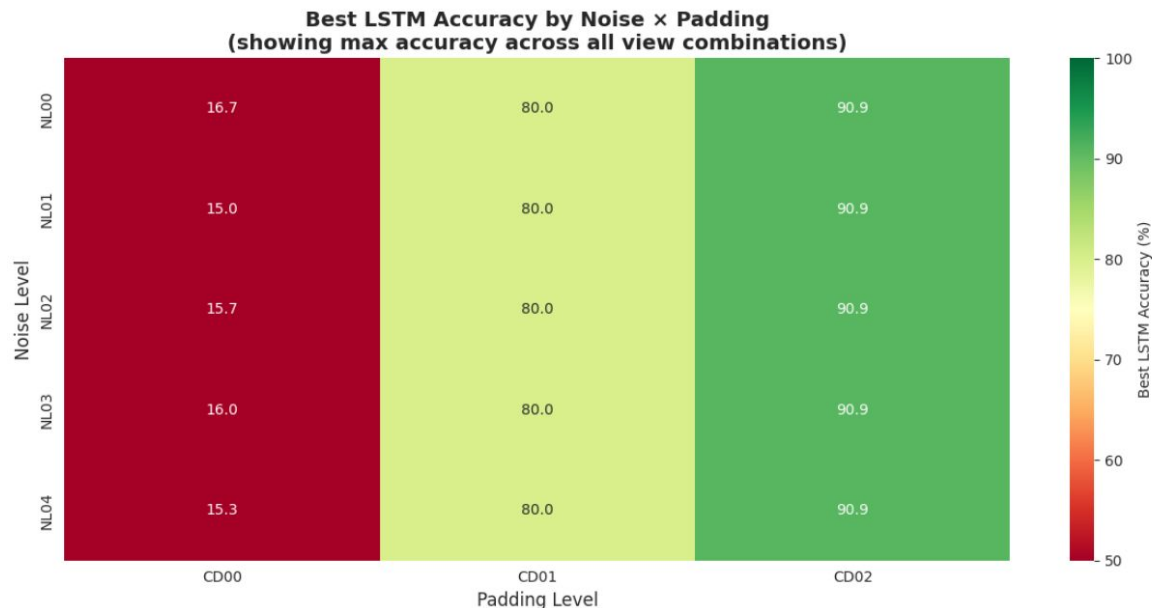
Distance Match + CNN Results cont.



Distance Match + CNN Results cont.

```
=====
NL00 + CD00: views_1_3_5 (93.45%)
NL00 + CD01: views_2_3_5 (99.94%)
NL00 + CD02: views_1_2 (99.98%)
NL01 + CD00: views_1_3_5 (90.22%)
NL01 + CD01: views_1_3_5_6 (99.37%)
NL01 + CD02: views_1_3 (98.83%)
NL02 + CD00: views_1_3_5 (88.62%)
NL02 + CD01: views_1_2_3_5_6 (98.10%)
NL02 + CD02: views_2_4 (98.08%)
NL03 + CD00: views_2_4_6 (85.28%)
NL03 + CD01: views_1_2_3_5_6 (96.76%)
NL03 + CD02: views_2_3_4 (96.82%)
NL04 + CD00: views_2_4_5_6 (65.45%)
NL04 + CD01: views_3_4_5 (90.11%)
NL04 + CD02: views_3_5 (92.03%)
=====
```

Part 5: LSTM with Sequences



Best view combinations used in heatmap above:

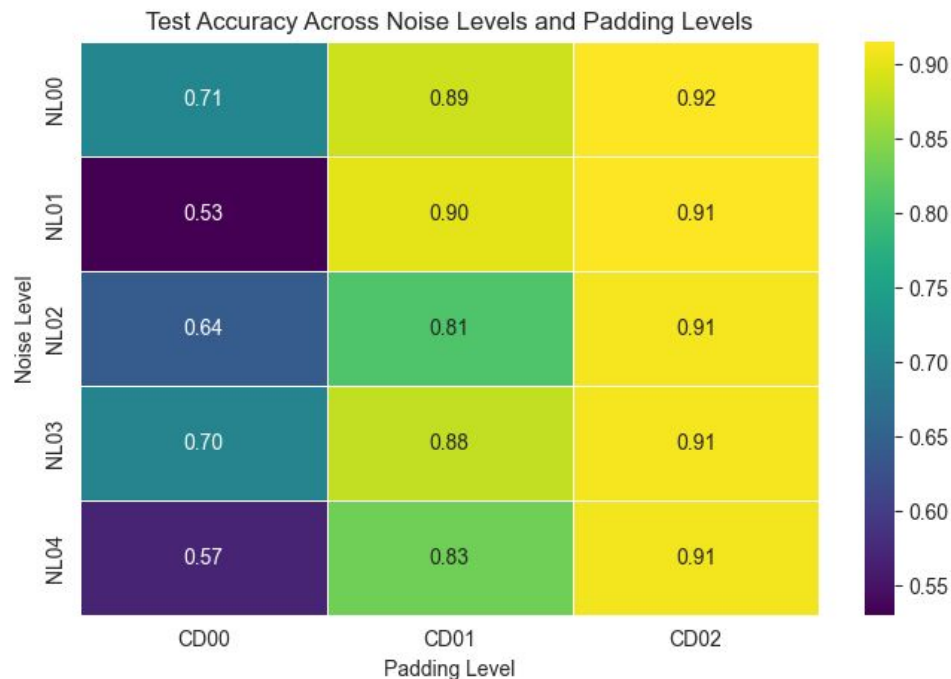
```
=====
NL00 + CD00: view_0 (16.73%)
NL00 + CD01: view_0 (80.00%)
NL00 + CD02: view_0 (90.91%)
NL01 + CD00: view_0 (14.97%)
NL01 + CD01: view_0 (80.01%)
NL01 + CD02: view_0 (90.91%)
NL02 + CD00: view_0 (15.71%)
NL02 + CD01: view_0 (80.00%)
NL02 + CD02: view_0 (90.91%)
NL03 + CD00: view_0 (16.03%)
NL03 + CD01: view_0 (80.00%)
NL03 + CD02: view_0 (90.91%)
NL04 + CD00: view_0 (15.30%)
NL04 + CD01: view_0 (80.02%)
NL04 + CD02: view_0 (90.91%)
=====
```

Best Combo of views (view 0) + padding + noise level.

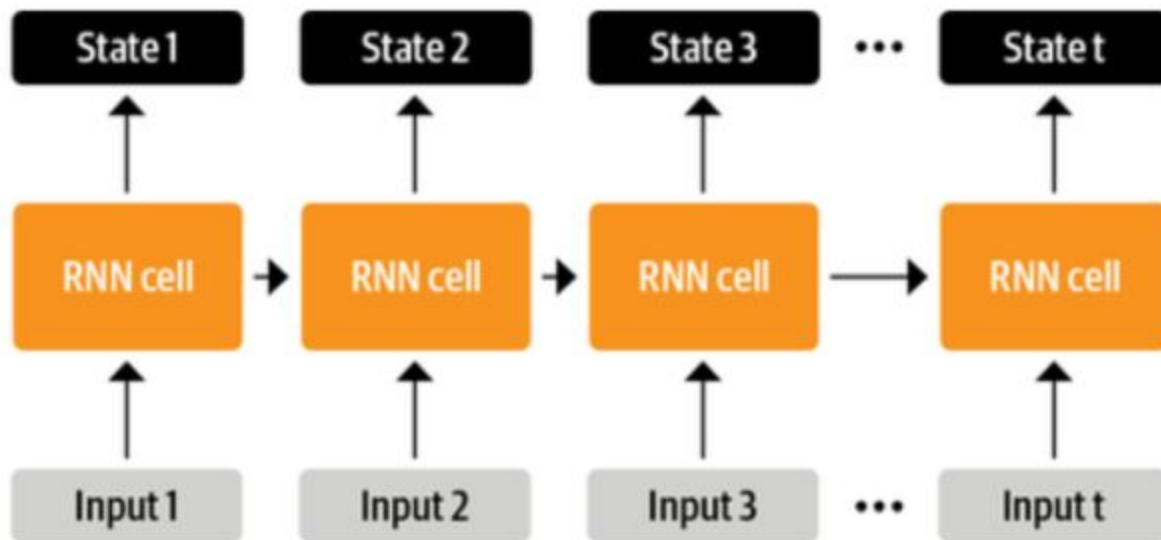
Key point: LSTM didn't work well and it might be due to it being "lazy"

Part 6: Baseline CNN

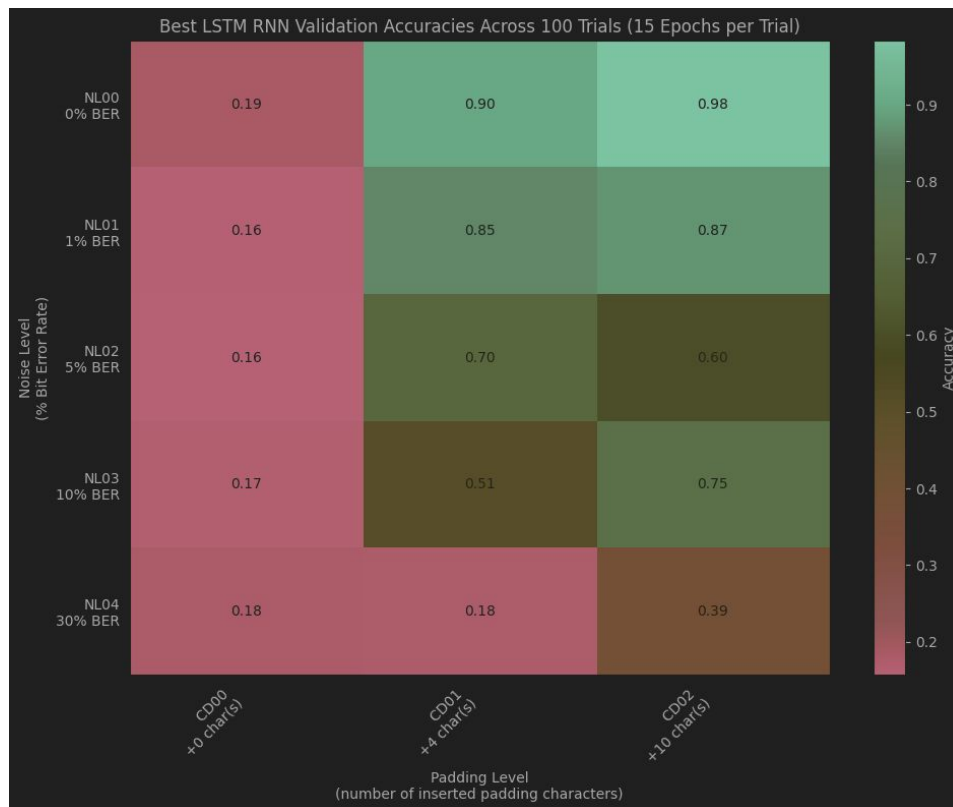
- CNN without Distance Layer
- Padding significantly improves accuracy
- >80% test accuracy on CD01 and CD02
- Highest accuracy on CD02
- Outperformed by CNN with Distance Layer



Recurrent Neural Networks (RNNs)



Baseline RNNs, LSTMs, GRUs



Connectionist Temporal Classification (CTC)

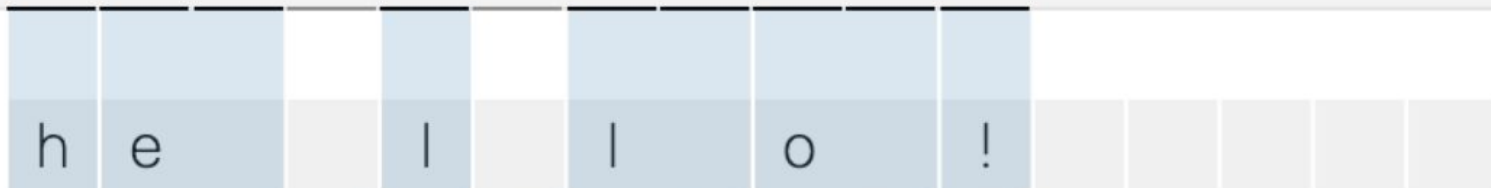
For an input,
like speech



Predict a
sequence of
tokens

h e e € | € | | o o !

Merge repeats,
drop €

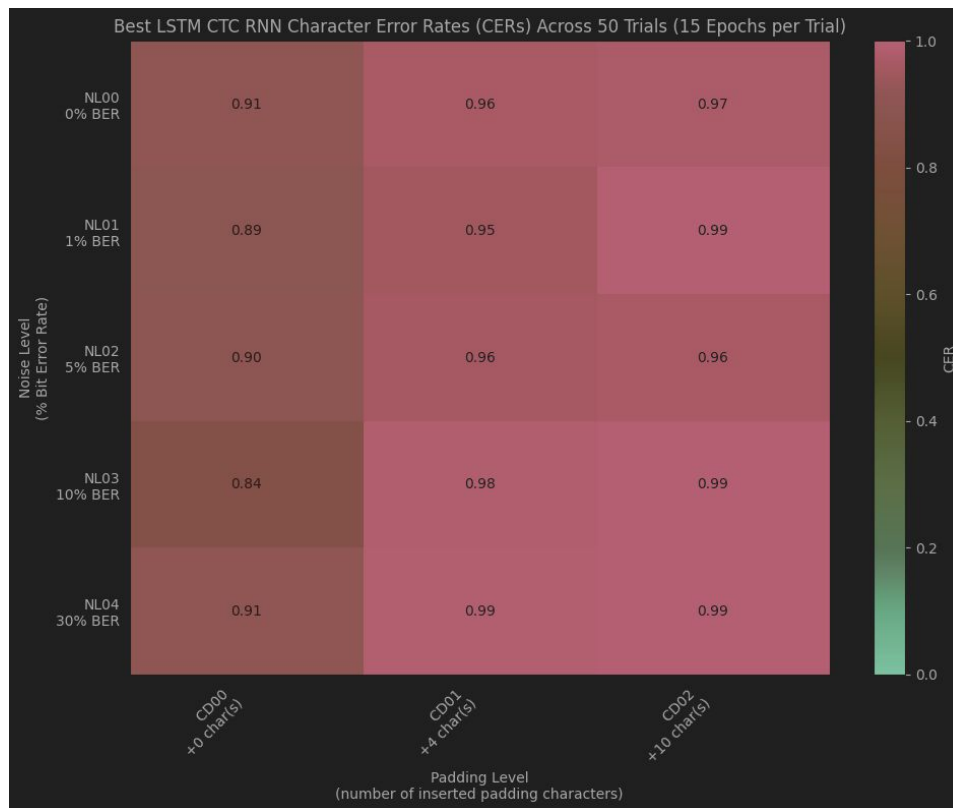


Final output

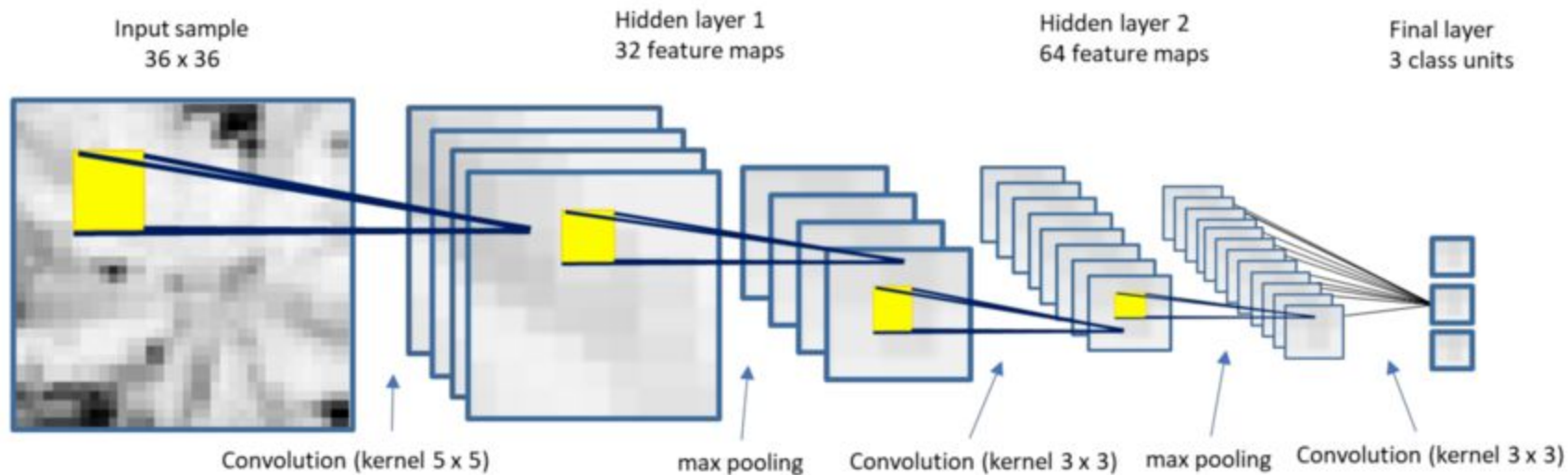


RNNs, LSTMs, GRUs + CTC Loss

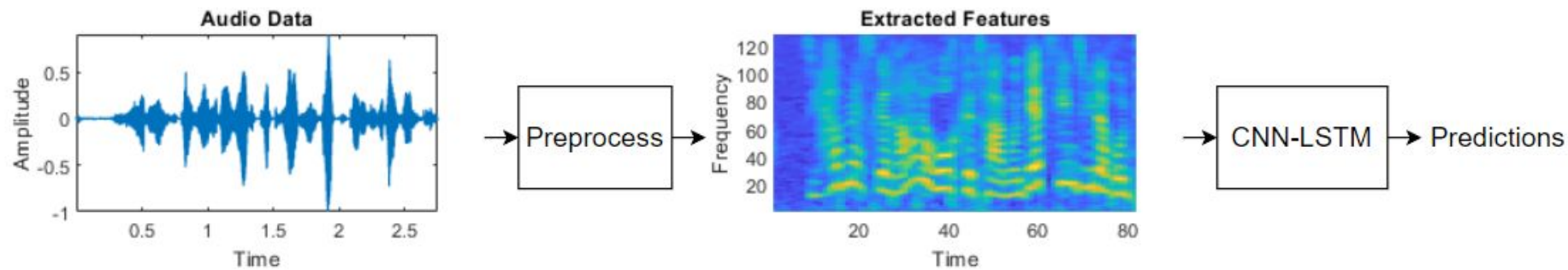
(Connectionist
Temporal
Classification)



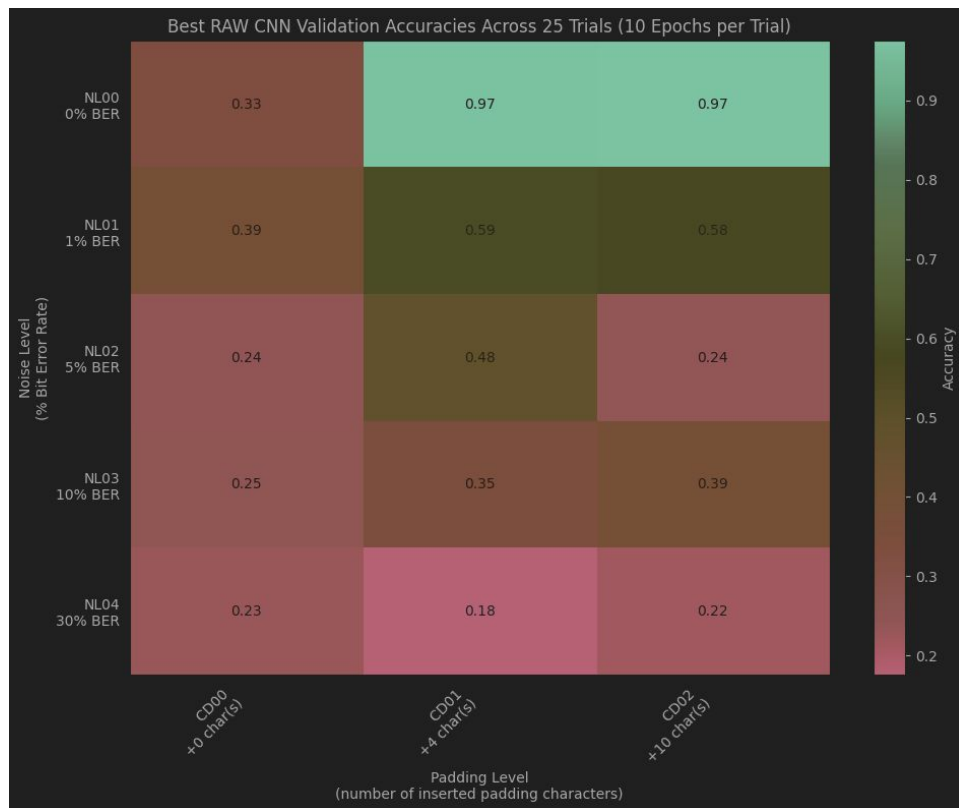
Convolutional Neural Networks



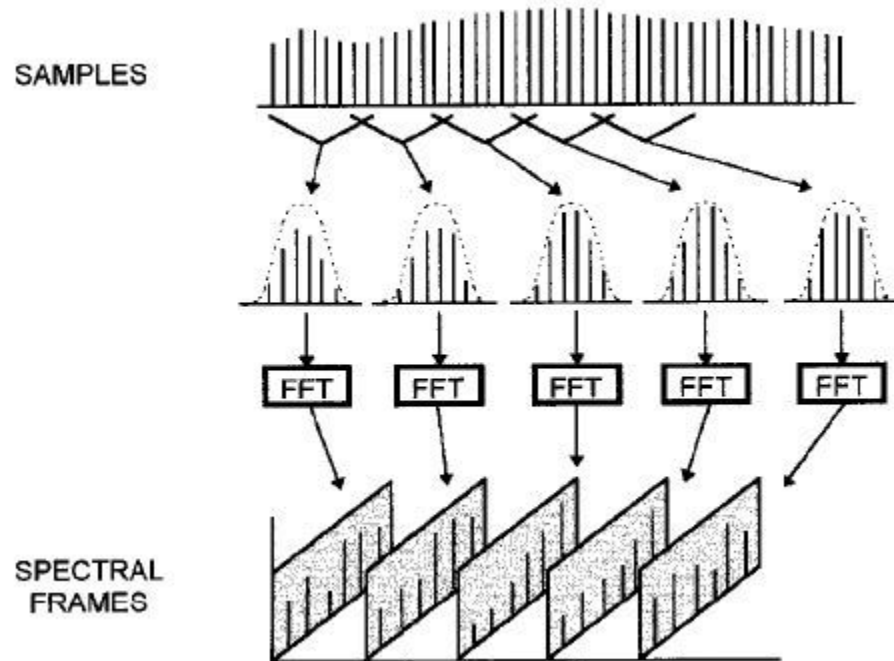
CNNs for Our Task



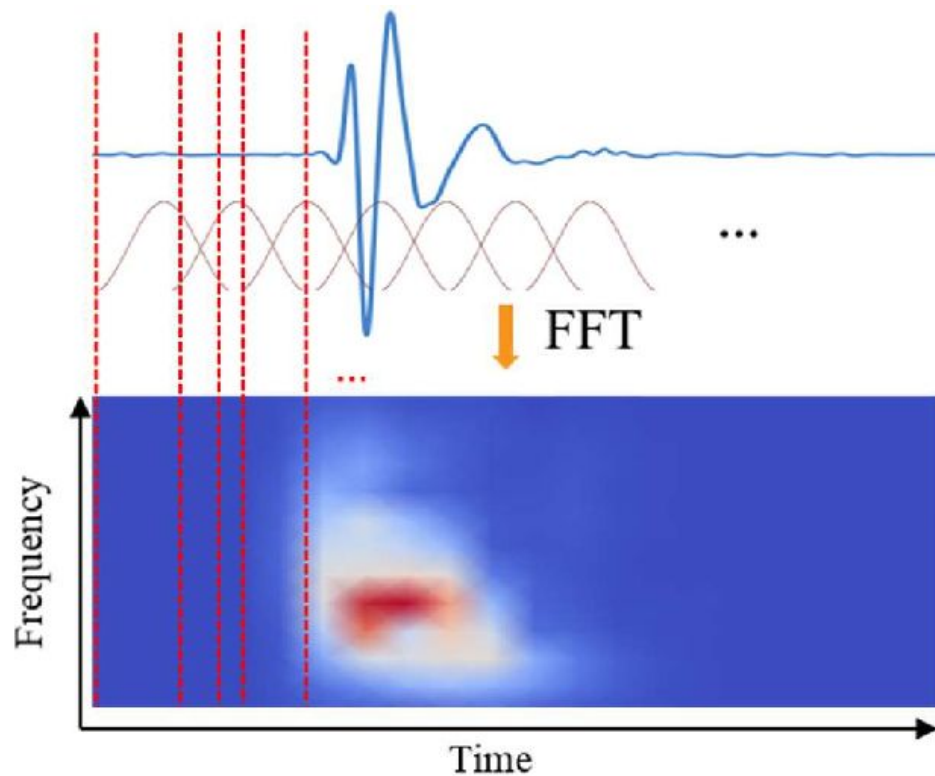
CNNs + Raw



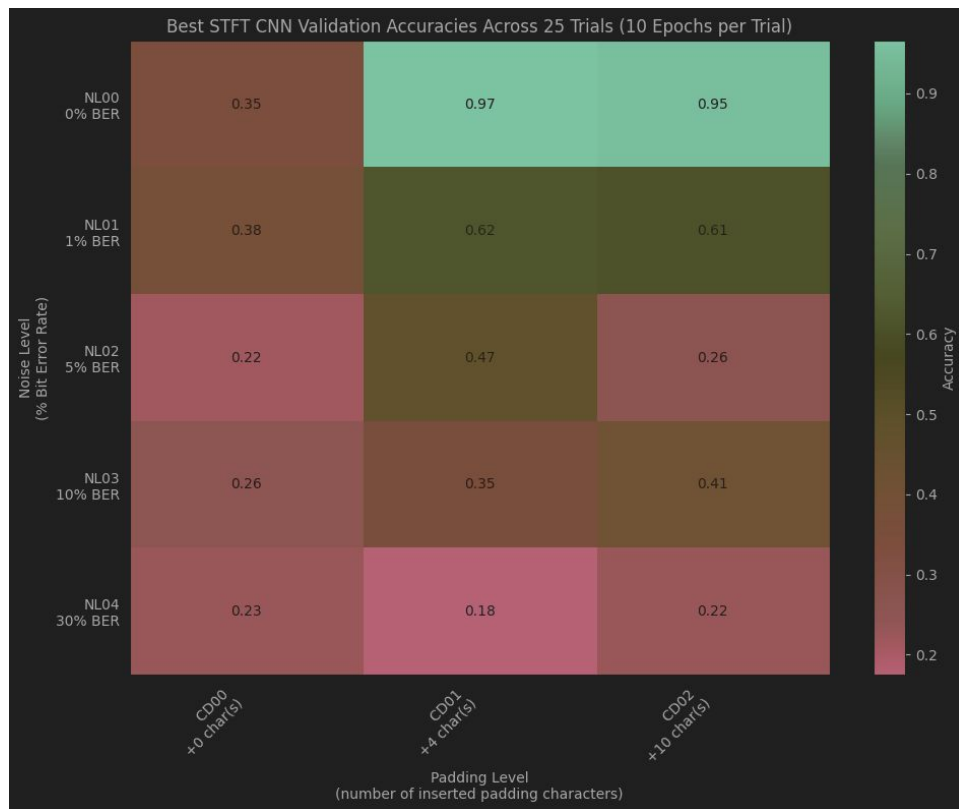
Short-Term Fourier Transform (STFT)



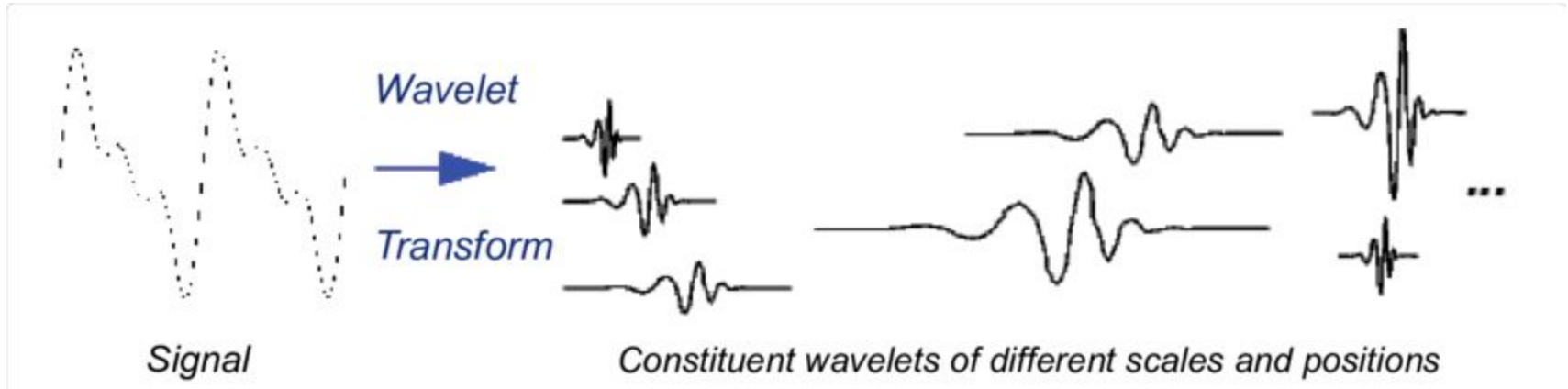
Short-Term Fourier Transform (STFT)



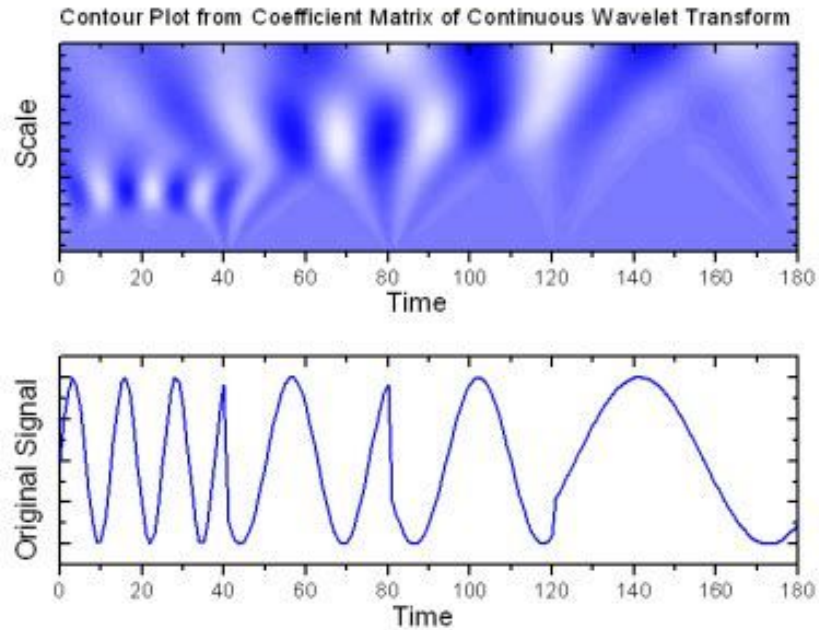
CNNs + Short-Term Fourier Transform (STFT)



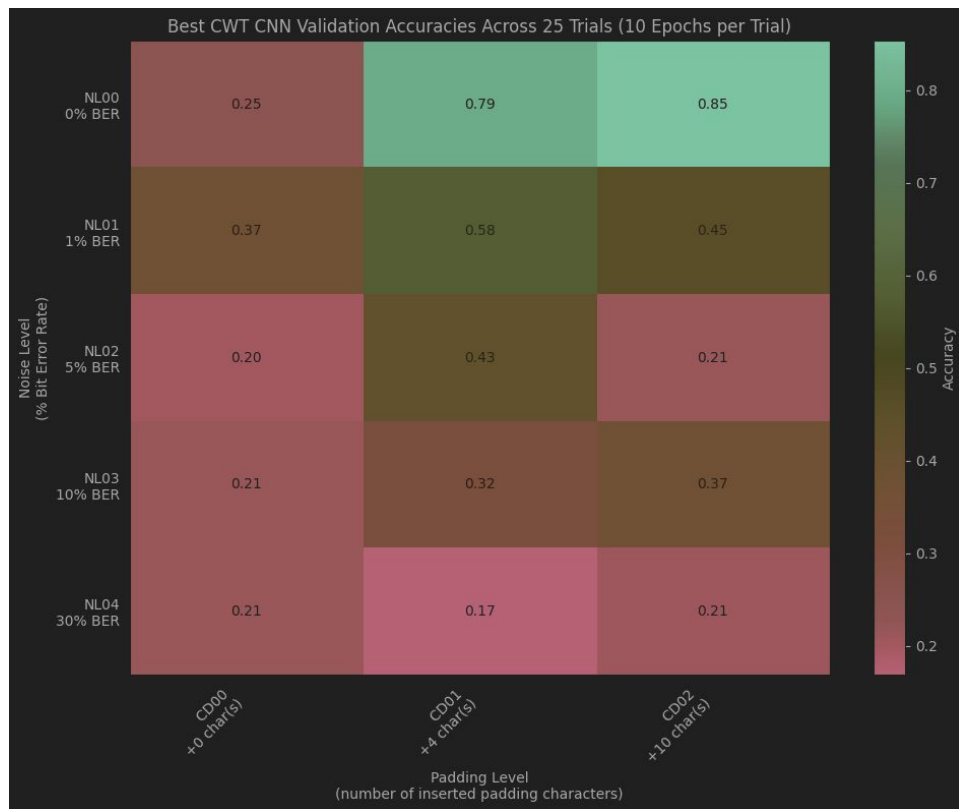
Continuous Wavelet Transform (CWT)



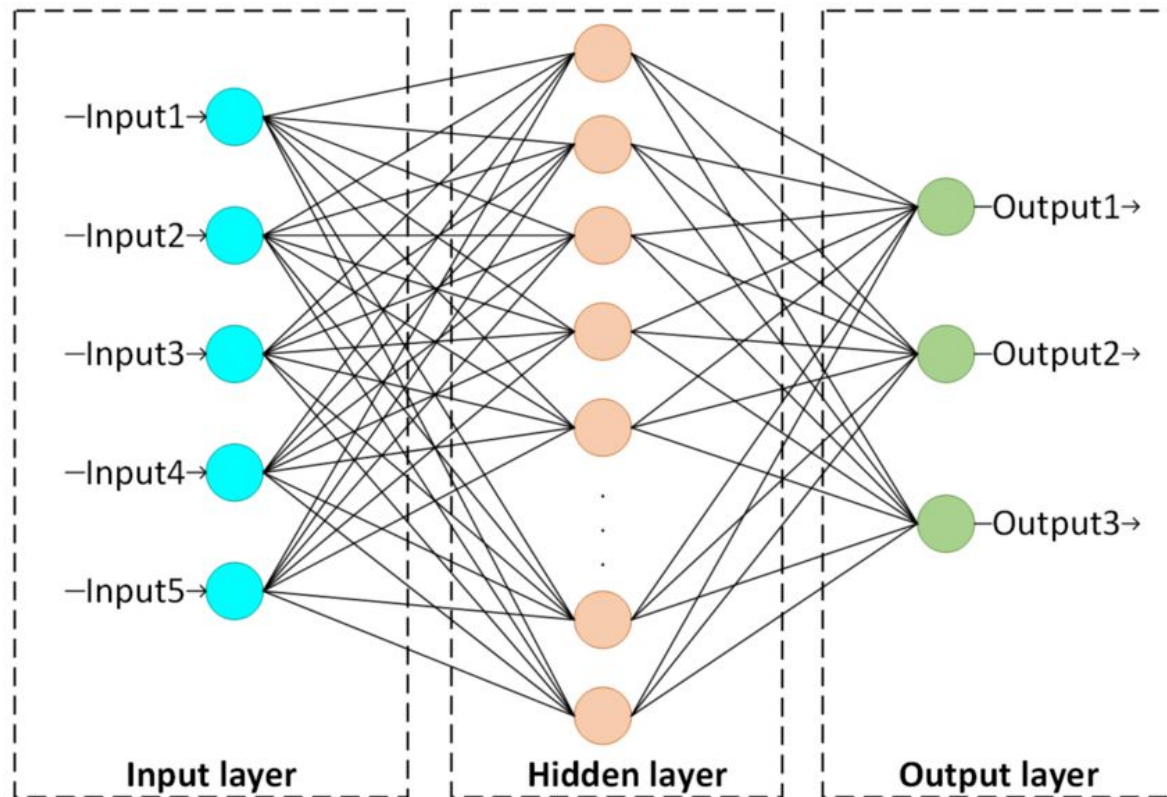
Continuous Wavelet Transform (CWT)



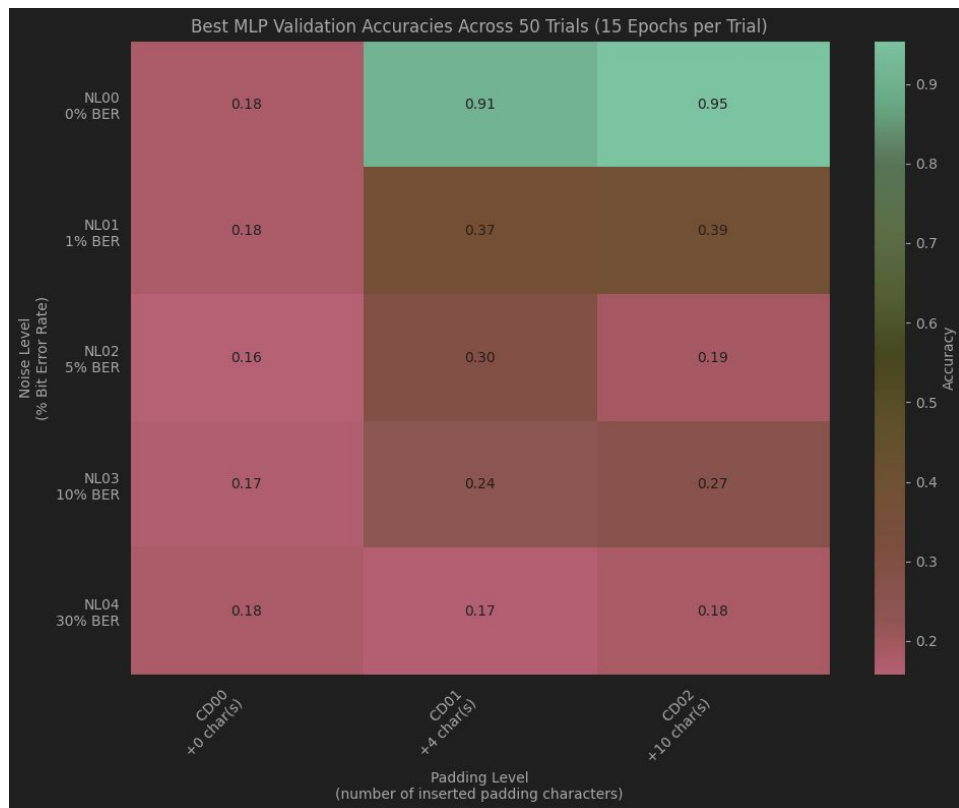
CNNs + Continuous Wavelet Transform (CWT)



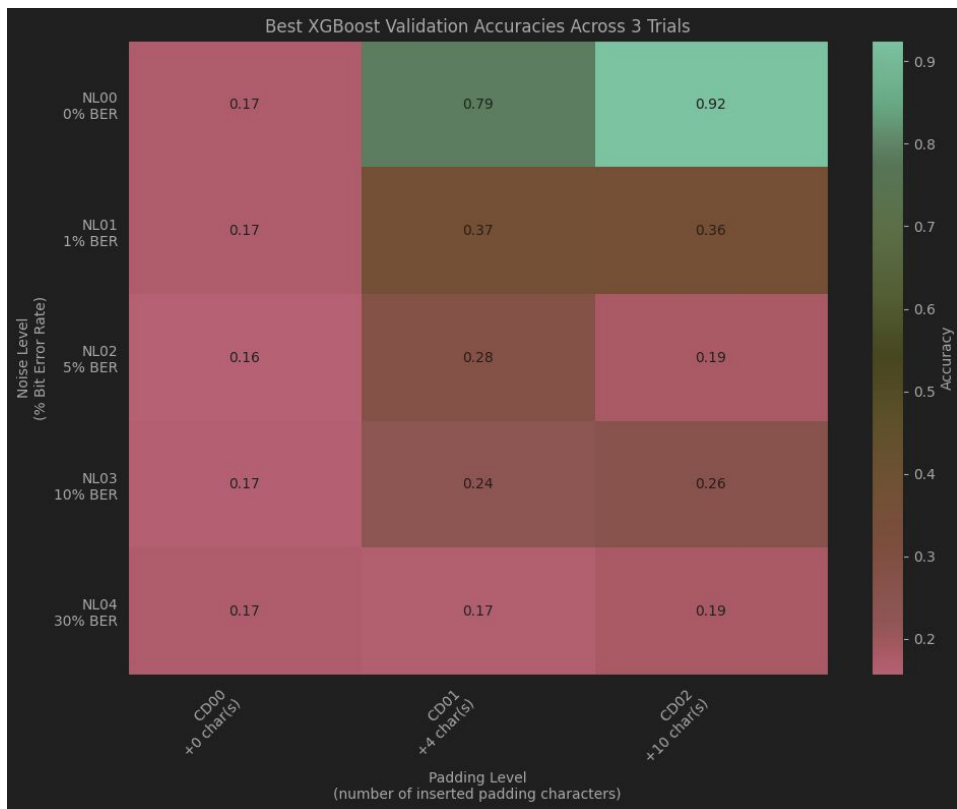
Multi-Layer Perceptrons (MLPs)



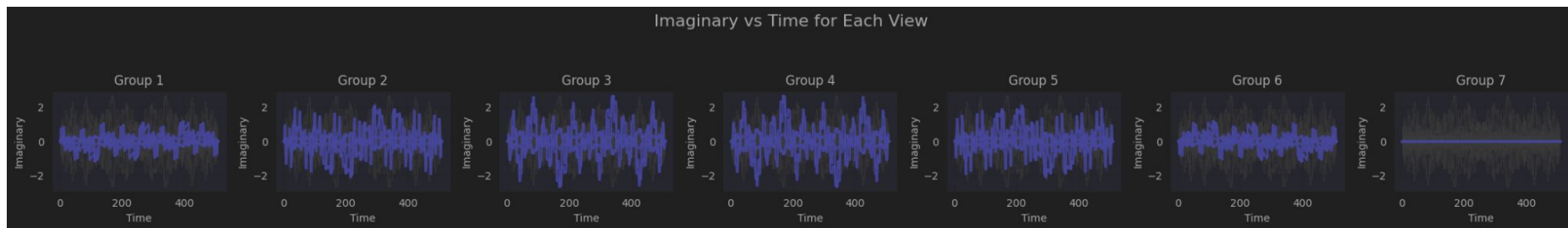
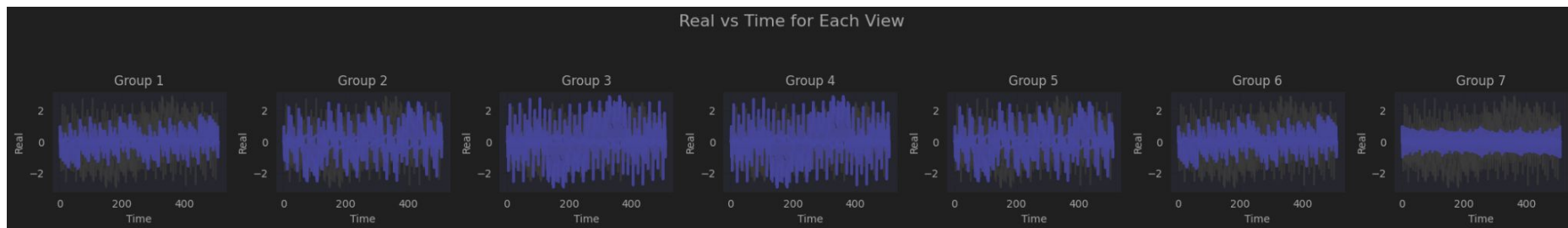
Baseline MLP



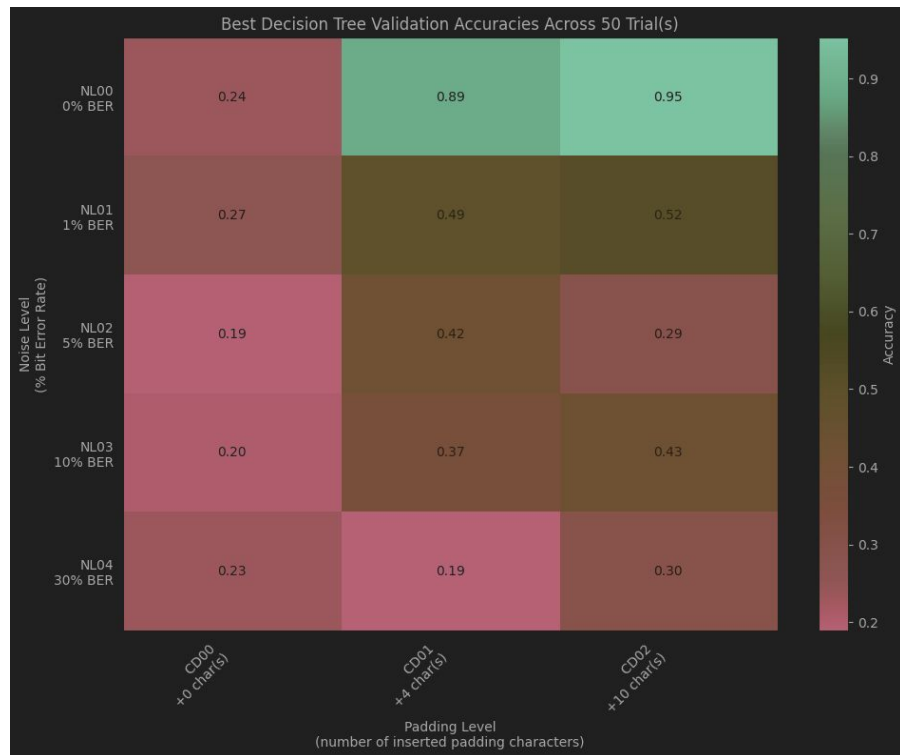
Traditional Supervised Learning: XGBoost



Views



Best Views: Decision Trees



Best view combination for Decision Tree_NL00_CD00: ('4', '5')

Best view combination for Decision Tree_NL00_CD01: ('2', '7')

Best view combination for Decision Tree_NL00_CD02: ('2', '3')

Best view combination for Decision Tree_NL01_CD00: ('4', '5')

Best view combination for Decision Tree_NL01_CD01: ('4', '5')

Best view combination for Decision Tree_NL01_CD02: ('4', '5')

Best view combination for Decision Tree_NL02_CD00: ('4', '5')

Best view combination for Decision Tree_NL02_CD01: ('4', '5')

Best view combination for Decision Tree_NL02_CD02: ('4', '5')

Best view combination for Decision Tree_NL03_CD00: ('4', '5')

Best view combination for Decision Tree_NL03_CD01: ('4', '5')

Best view combination for Decision Tree_NL03_CD02: ('4', '5')

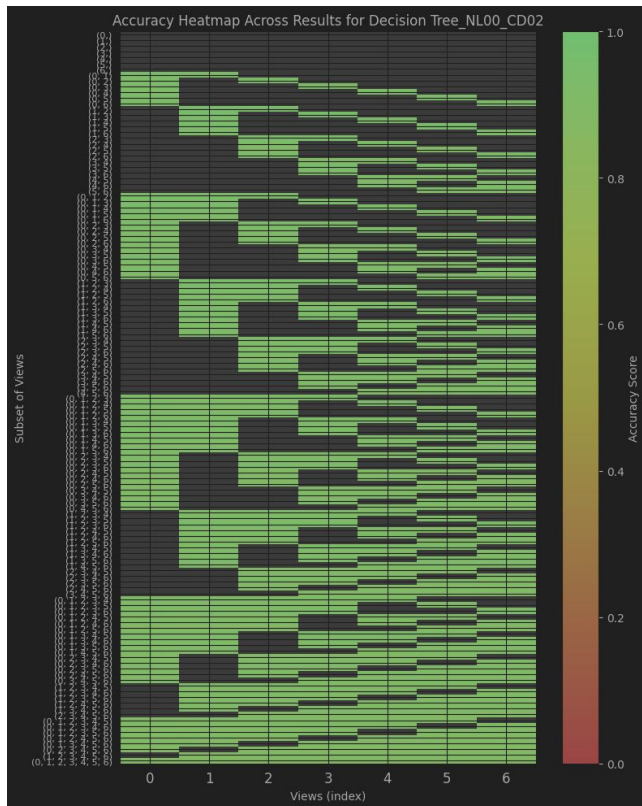
Best view combination for Decision Tree_NL04_CD00: ('4', '5')

Best view combination for Decision Tree_NL04_CD01: ('3', '4', '5')

Best view combination for Decision Tree_NL04_CD02: ('4', '5')

(1-indexed)

Best Views: Decision Trees

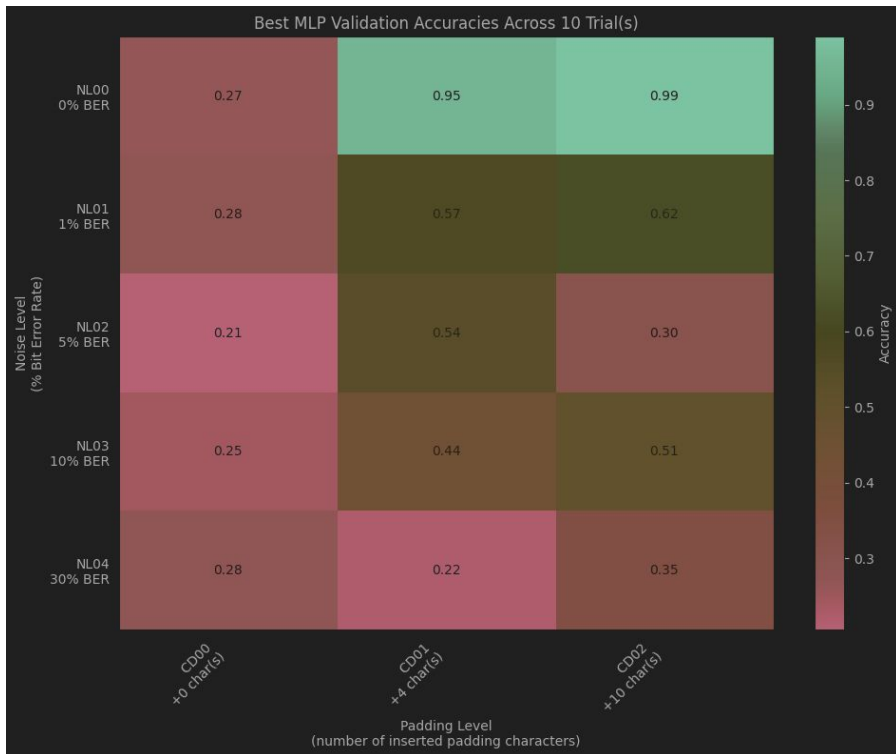


Top 10 Accuracies for Results of Decision Tree_NL00_CD02

- 1: ('2', '3') -- 95.3% accuracy
- 2: ('2', '4') -- 95.1% accuracy
- 3: ('2', '6') -- 94.7% accuracy
- 4: ('2', '3', '4') -- 94.4% accuracy
- 5: ('1', '3') -- 93.7% accuracy
- 6: ('1', '2') -- 93.5% accuracy
- 7: ('2', '5') -- 93.5% accuracy
- 8: ('1', '2', '4') -- 93.5% accuracy
- 9: ('2', '3', '5') -- 93.4% accuracy
- 10: ('4', '6') -- 93.3% accuracy

(1-indexed)

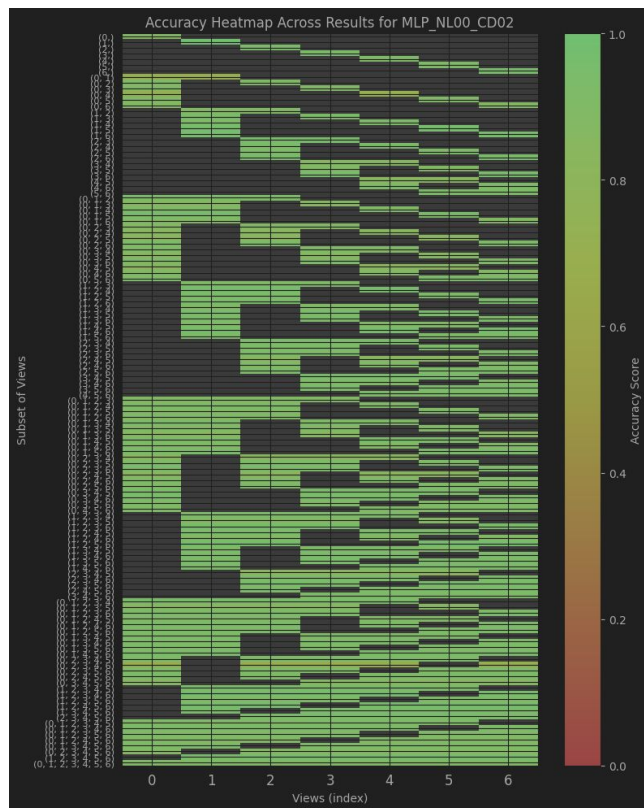
Best Views: MLPs



Best view combination for MLP_NL00_CD00: ('5',)
Best view combination for MLP_NL00_CD01: ('7',)
Best view combination for MLP_NL00_CD02: ('2',)
Best view combination for MLP_NL01_CD00: ('5',)
Best view combination for MLP_NL01_CD01: ('5',)
Best view combination for MLP_NL01_CD02: ('5',)
Best view combination for MLP_NL02_CD00: ('4',)
Best view combination for MLP_NL02_CD01: ('5',)
Best view combination for MLP_NL02_CD02: ('4',)
Best view combination for MLP_NL03_CD00: ('5',)
Best view combination for MLP_NL03_CD01: ('4',)
Best view combination for MLP_NL03_CD02: ('5',)
Best view combination for MLP_NL04_CD00: ('5',)
Best view combination for MLP_NL04_CD01: ('4',)
Best view combination for MLP_NL04_CD02: ('4',)

(1-indexed)

Best Views: MLPs

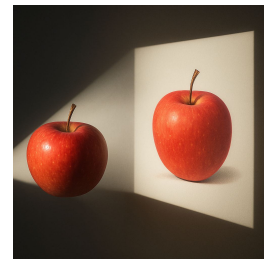
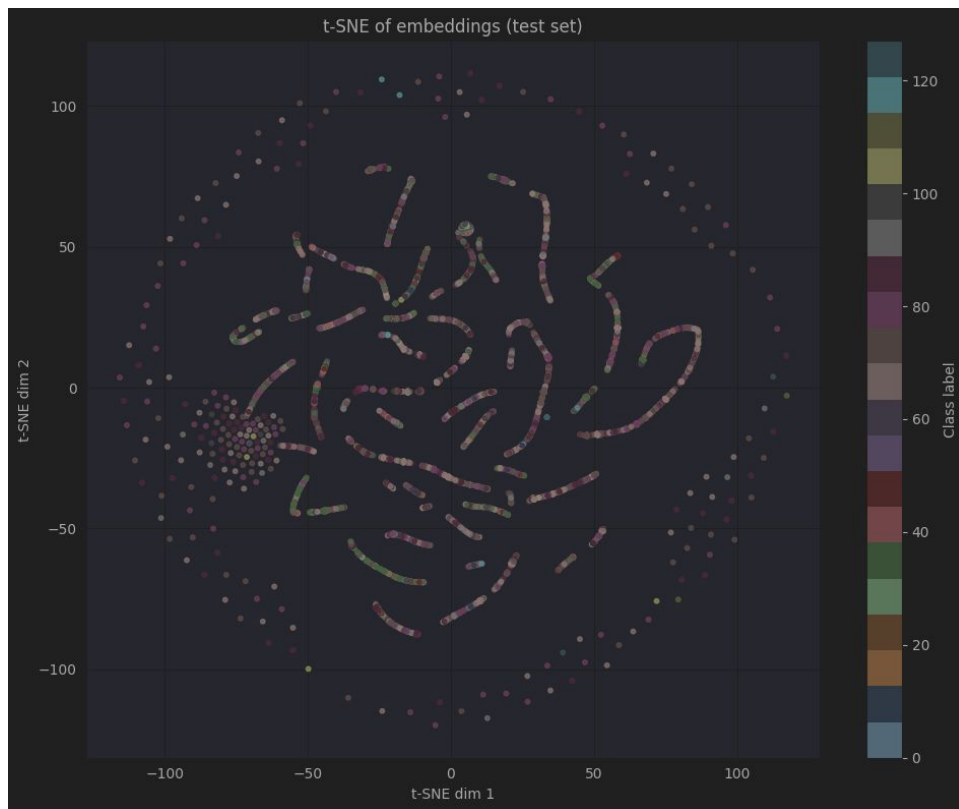


Top 10 Accuracies for Results of MLP_NL00_CD02

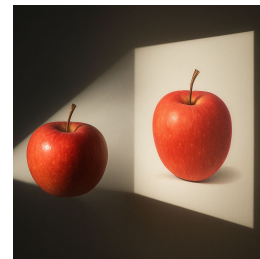
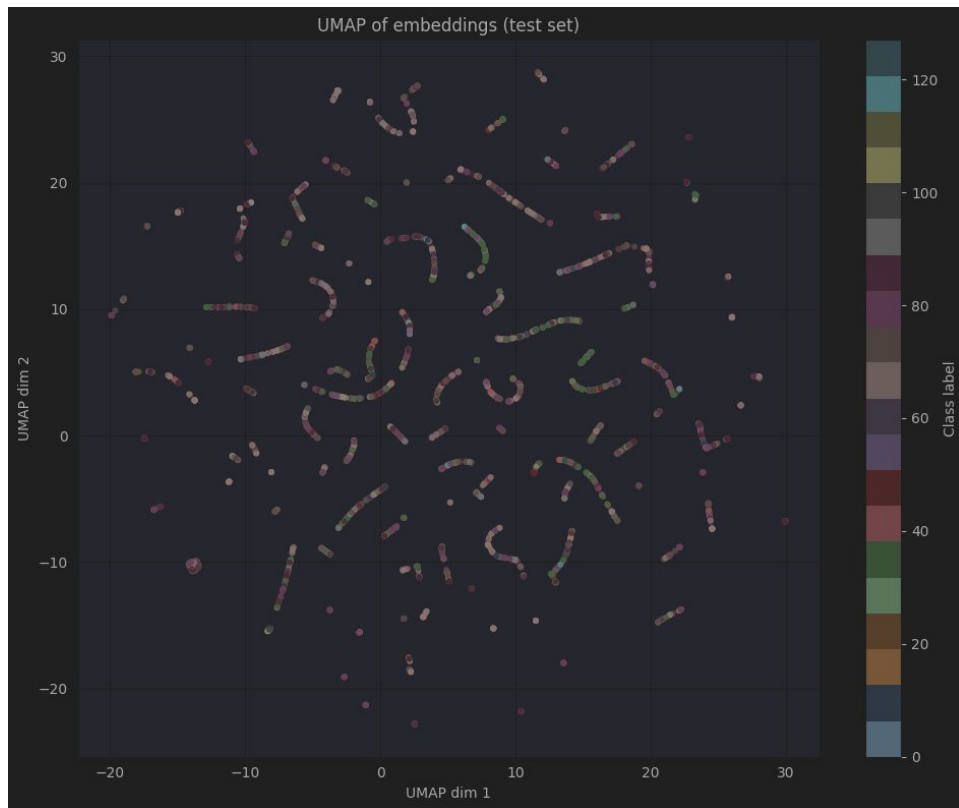
- 1: ('2',) -- 99.0% accuracy
- 2: ('2', '3', '6', '7') -- 96.7% accuracy
- 3: ('2', '6') -- 96.6% accuracy
- 4: ('2', '3', '5') -- 96.0% accuracy
- 5: ('2', '5', '7') -- 96.0% accuracy
- 6: ('2', '3', '4', '5', '7') -- 95.9% accuracy
- 7: ('2', '7') -- 95.8% accuracy
- 8: ('2', '3', '5', '6', '7') -- 95.8% accuracy
- 9: ('4', '5', '6') -- 95.2% accuracy
- 10: ('2', '4', '6') -- 95.1% accuracy

(1-indexed)

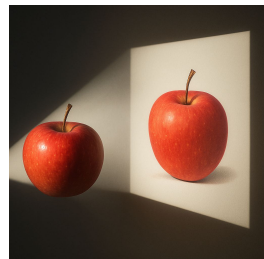
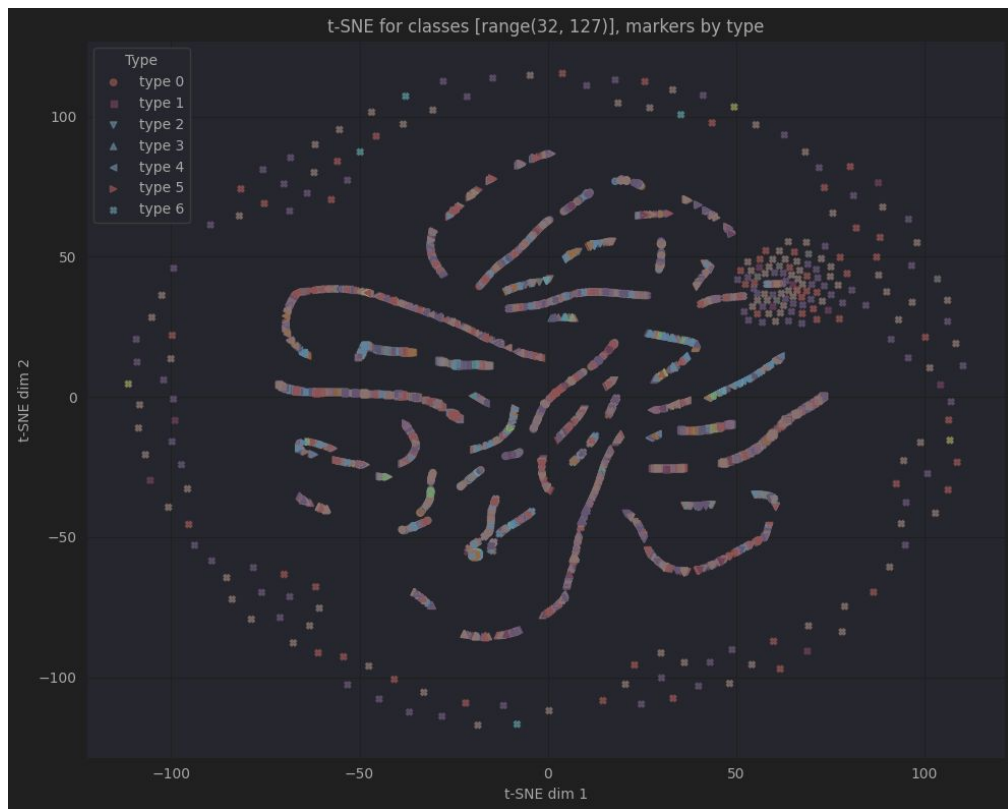
Contrastive Learning: TSNE Projection



Contrastive Learning: UMAP Projection



Contrastive Learning: TSNE Projection with Views



Issues/Challenges

Primary Roadblock

- Temporary government shutdown created communication gap

Impact on Methodology

- Strategy Misalignment
- Corrected approach late in the semester

Resolution

- Realignment and Success

Future Work

Key Observations

- Irregular Scaling
- Counter-Intuitive

Next Steps

- Hyperparameter Tuning
- Feature Importance

Conclusion

Primary Achievement

- Feasibility Confirmed

Key Technical Findings

- Top Performer (Distance + CNN)
- LSTM Limitations
- Data Scaling

Bibliography

Papers:

- [1] Agarwal, A. (2019, June 11). Signal Modeling Using Recurrent Neural Networks. Retrieved September 12, 2025, from Medium website: <https://pub.towardsai.net/signal-modelling-using-recurrent-neural-networks-d832f0c50503>
- [2] Kiranyaz, S. (2021). 1D convolutional neural networks and applications: A survey. *Mechanical Systems and Signal Processing*, 151, 107398. <https://doi.org/10.1016/j.ymssp.2020.107398>
- [3] Shevitski, B., Watkins, Y., Man, N., & Girard, M. (2023). Digital Signal Processing Using Deep Neural Networks. *OSTI OAI (U.S. Department of Energy Office of Scientific and Technical Information)*. <https://doi.org/10.2172/1984848>

Acronyms

AI – Artificial Intelligence

ML – Machine Learning

RF – Radio Frequency

NN – Neural Network

CNN – Convolutional Neural Network

RNN – Recurrent Neural Network

LSTM – Long Short-Term Memory

I/Q – In-phase / Quadrature

DOE – Department of Energy

PNNL – Pacific Northwest National Laboratory

UTD – The University of Texas at Dallas

FGSM – Fast Gradient Sign Method

GAN – Generative Adversarial Network

AWS – Amazon Web Services

PKI – Public Key Infrastructure